

Seguridad de la capa de transporte

Seguridad de la capa de transporte (en inglés: *Transport Layer Security*, **TLS**) y su antecesor **Secure Sockets Layer** (**SSL**; en español **capa de puertos seguros**) son protocolos criptográficos, que proporcionan comunicaciones seguras por una red, comúnmente Internet.¹ El protocolo TLS es el sucesor de SSL (1995), introducido en 1999 con una versión mejorada de SSL 3.0 (que al principio se llamó SSL 3.1). La versión actual es TLS 1.3 (a partir de 2018). Las versiones de SSL están desactualizadas y se consideran inseguras.² Se usan certificados X.509, y por lo tanto criptografía asimétrica para autenticar a la contraparte con quien se están comunicando,³ y para intercambiar una clave simétrica. Esta sesión es luego usada para cifrar el flujo de datos entre las partes. Esto permite la confidencialidad del mensaje, códigos de autenticación de mensajes para integridad y como un producto lateral, autenticación del mensaje. Varias versiones del protocolo están en aplicaciones ampliamente utilizadas como navegación web, correo electrónico, fax por Internet, mensajería instantánea y voz-sobre-IP (VoIP). Una propiedad importante en este contexto es *forward secrecy*, para que la clave de corta vida de la sesión no pueda ser descubierta a partir de la clave asimétrica de largo plazo.⁴

TLS es un protocolo de Internet Engineering Task Force (IETF), definido por primera vez en 1999, que fue actualizado en el RFC 5246 (agosto de 2008) y en RFC 6176 (marzo de 2011). Se basa en las especificaciones previas de SSL (1994, 1995, 1996) desarrolladas por Netscape Communications⁵ para agregar el protocolo HTTPS a su navegador Netscape Navigator. Su última versión, TLS 1.3, fue definida en agosto de 2018.⁶

Seguridad de la capa de transporte

Familia	Internet
Función	Seguridad en la capa de transporte
Última versión	1.3

Ubicación en la pila de protocolos

<i>Aplicación</i>	HTTPS, IMAPS, POP3S, SMTPS, ...
Transporte	TLS
	TCP
<i>Red</i>	IP
	Estándares
	RFC 8446, RFC 5705, RFC 6066, otros

Descripción

SSL proporciona autenticación y privacidad de la información entre extremos sobre Internet mediante el uso de criptografía. Habitualmente, solo el servidor es autenticado (es decir, se garantiza su identidad) mientras que el cliente se mantiene sin autenticar.

SSL implica una serie de fases básicas:

- Negociar entre las partes el algoritmo que se usará en la comunicación
- Intercambio de claves públicas y autenticación basada en certificados digitales.
- Cifrado del tráfico basado en cifrado simétrico.

Durante la primera fase, el cliente y el servidor negocian qué algoritmos criptográficos se van a usar. Las implementaciones actuales proporcionan las siguientes opciones:

- Para criptografía de clave pública: RSA, Diffie-Hellman, DSA (*Digital Signature Algorithm*) o Fortezza.
- Para cifrado simétrico: RC2, RC4, IDEA (*International Data Encryption Algorithm*), DES (*Data Encryption Standard*), Triple DES y AES (*Advanced Encryption Standard*).
- Con funciones hash: MD5 o de la familia SHA.

Historia y desarrollo

API de Secure Network Programming

Los primeros esfuerzos de investigación hacia la seguridad de la capa de transporte incluyeron la interfaz de programación de aplicaciones (API, por su sigla en inglés) de Secure Network Programming (SNP), la que en 1993 exploró la posibilidad de tener una API de capa de transporte segura similar a los sockets Berkeley, para facilitar la retroadaptación de las aplicaciones de red preexistentes con medidas de seguridad.⁷

SSL 3.0

El protocolo SSL fue desarrollado originalmente por Netscape.⁸ La versión 1.0 nunca se entregó públicamente; la versión 2.0 se presentó en febrero de 1995 pero "contenía una cantidad de fallas de seguridad que al final llevaron al diseño de la versión SSL 3.0".⁹ Dicha versión, presentada en 1996, fue un rediseño completo del protocolo producido por Paul Kocher, quien trabajó con los ingenieros de Netscape Phil Karlton y Alan Freier. Las versiones más nuevas de SSL/TLS están basadas en SSL 3.0. El borrador de 1996 de SSL 3.0 fue publicado por la IETF como el histórico RFC 6101. En octubre de 2014, se detectó una nueva vulnerabilidad sobre el protocolo SSL en su versión 3.0, la Vulnerabilidad de Poodle.

Protocolo	Publicación
SSL 1.0	No publicado
SSL 2.0	1995
SSL 3.0	1996
TLS 1.0	1999
TLS 1.1	2006
TLS 1.2	2008
TLS 1.3	2018

TLS 1.0

TLS 1.0 fue definido en el RFC 2246 en enero de 1999 y es una actualización de SSL versión 3.0. Como dice el RFC, "las diferencias entre este protocolo y SSL 3.0 no son dramáticas, pero son suficientemente significativas como para impedir la interoperabilidad entre TLS 1.0 y SSL 3.0". TLS 1.0 incluye una forma en la cual la implementación puede conectarse en SSL 3.0, debilitando la seguridad.

TLS 1.1

TLS 1.1 fue definido en el RFC 4346 en abril de 2006.¹⁰ Es una actualización de TLS 1.0. Las diferencias más significativas incluyen:

- Agrega protección contra ataques de CBC.
 - El vector de inicialización (IV) implícito fue reemplazado por un IV explícito.
 - Cambio en el manejo de los errores de relleno.
- Soporte para el registro de parámetros de IANA.

TLS 1.2

TLS 1.2 fue definido originalmente en el RFC 5246 en agosto del 2008. Se basa en una especificación posterior de TLS 1.1. Las mayores diferencias son:

- la combinación MD5-SHA-1 en la función pseudoaleatoria (PRF) fue reemplazada por SHA-256, con la opción de usar las PRF especificadas en la *cipher-suite*.
- la combinación MD5-SHA-1 en el mensaje terminado fue reemplazada por SHA-256, sin la opción de usar algoritmos de *hash* específicos para la *cipher-suite*. Sin embargo, el tamaño del *hash* en el mensaje terminado es truncado a 96 bits.
- la combinación MD5-SHA-1 en el elemento digitalmente firmado fue reemplazada por un *hash* simple negociado durante el *handshake*, que por defecto es SHA-1.
- Mejoras en la habilidad de clientes y servidores para especificar qué algoritmos de *hash* y de firma van a aceptar.
- Expansión del soporte de cifras de cifrado autenticadas, usadas mayormente para modo Galois/Counter (GCM) y modo CCM del cifrado con Advanced Encryption Standard (AES).
- Se agregaron definición de Extensiones de TLS y de Ciphersuites de AES.

TLS 1.2 fue después redefinido en el RFC 6176 de marzo de 2011 redactando su retrocompatibilidad con SSL y TLS para que dichas sesiones jamás negocien el uso de SSL versión 2.0.

TLS 1.3

TLS 1.3 fue definido en el RFC 8446 en agosto de 2018. Está basado en la anterior especificación TLS 1.2. Las principales diferencias con TLS 1.2 incluyen:

- Un modo 0-RTT
- Retiro de la hora GMT.
- Fusiona soporte de ECC del RFC 4492 pero sin curvas explícitas.
- Retira el campo de longitud innecesaria de la entrada de AD a cifras AEAD.
- Cambiar el nombre de {Cliente, Servidor} KeyExchange a {Cliente, Servidor} KeyShare
- Añade un *HelloRetryRequest* explícita para rechazar el del cliente
- *Handshake* revisado a fin de proporcionar el modo 1-RTT.
- Retiro de grupos DHE personalizados.
- Eliminado el soporte para la compresión.
- Eliminado el soporte para el intercambio de claves RSA estática y DH.
- Eliminado el soporte para sistemas de cifrado no AEAD.

Funcionamiento

El protocolo SSL intercambia registros; opcionalmente, cada registro puede ser comprimido, cifrado y empaquetado con un código de autenticación del mensaje (MAC). Cada registro tiene un campo de *content_type* que especifica el protocolo de nivel superior que se está usando.

Cuando se inicia la conexión, el nivel de registro encapsula otro protocolo, el protocolo *handshake* (o protocolo de acuerdo), que tiene el *content_type* 22.

El cliente envía y recibe varias estructuras *handshake*:

- Envía un mensaje *ClientHello* especificando una lista de conjunto de cifrados, métodos de compresión y la versión del protocolo SSL más alta permitida. Este también envía bytes aleatorios que serán usados más tarde (llamados *Challenge de Cliente* o *Reto*). Además puede incluir el identificador de la sesión.
- Después, recibe un registro *ServerHello*, en el que el servidor elige los parámetros de conexión a partir de las opciones ofertadas con anterioridad por el cliente.
- Cuando los parámetros de la conexión son conocidos, cliente y servidor intercambian certificados (dependiendo de las claves públicas de cifrado seleccionadas). Estos certificados son actualmente X.509, pero hay también un borrador especificando el uso de certificados basados en OpenPGP.¹¹
- Cliente y servidor negocian una clave secreta (simétrica) común llamada *master secret*, posiblemente usando el resultado de un intercambio Diffie-Hellman, o simplemente cifrando una clave secreta con una clave pública que es descifrada con la clave privada de cada uno. Todos los datos de claves restantes son derivados a partir de este *master secret* (y los valores aleatorios generados en el cliente y el servidor), que son pasados a través una *función pseudoaleatoria* cuidadosamente elegida.

TLS/SSL poseen una variedad de medidas de seguridad:

- Numerando todos los registros y usando el número de secuencia en el MAC.
- Usando un resumen de mensaje mejorado con una clave (de forma que solo con dicha clave se pueda comprobar el MAC). Esto se especifica en el RFC 2104.
- Protección contra varios ataques conocidos (incluyendo ataques *man-in-the-middle*), como los que implican un degradado del protocolo a versiones previas (por tanto, menos seguras), o conjuntos de cifrados más débiles.
- El mensaje que finaliza el protocolo *handshake* (*Finished*) envía un *hash* de todos los datos intercambiados y vistos por ambas partes.
- La función pseudo aleatoria divide los datos de entrada en 2 mitades y las procesa con algoritmos *hash* diferentes (MD5 y SHA), después realiza sobre ellos una operación XOR. De esta forma se protege a sí mismo de la eventualidad de que alguno de estos algoritmos se revelen vulnerables en el futuro.

Intercambio de claves

Antes de que un cliente y el servidor pueden empezar a intercambiar información protegida por TLS, deben intercambiar en forma segura o acordar una clave de cifrado y una clave para usar cuando se cifren los datos (ver Cifrado). Entre los métodos utilizados para el intercambio/acuerdo de claves son: las claves públicas y privadas generadas con RSA (denotado TLS_RSA en el protocolo de *handshake* TLS), Diffie-Hellman (llamado TLS_DH), Diffie-Hellman efímero (denotado TLS_DHE), Diffie-Hellman de Curva Elíptica (denotado TLS_ECDH), Diffie-Hellman de Curva Elíptica efímero (TLS_ECDHE), Diffie-Hellman anónimo (TLS_DH_anon),³ y PSK (TLS_PSK).¹²

El método de acuerdo de claves TLS_DH_anon no verifica el servidor o el usuario y por lo tanto rara vez se utiliza puesto que es vulnerable a un ataque de suplantación de identidad. Solo TLS_DHE y TLS_ECDHE proporcionan secreto-perfecto-hacia-adelante.

Los certificados de clave pública que se utilizan durante el intercambio/acuerdo también varían en el tamaño de las claves de cifrado públicas/privadas utilizadas durante el intercambio y, por tanto, en la solidez de la seguridad que proveen. En julio de 2013, Google anunció que dejaría de utilizar claves públicas 1024 bits y cambiaría a claves de 2048 bits para aumentar la seguridad del cifrado TLS que proporciona a sus usuarios.¹³

Autenticación e Intercambio/acuerdo de claves

Algoritmo	SSL 2.0	SSL 3.0	TLS 1.0	TLS 1.1	TLS 1.2	TLS 1.3	Estatus
<u>RSA</u>	Sí	Sí	Sí	Sí	Sí		Definido para TLS 1.2 en RFCs
<u>DH-RSA</u>	No	Sí	Sí	Sí	Sí		
<u>DHE-RSA (forward secrecy)</u>							
<u>ECDH-RSA</u>	No	No	Sí	Sí	Sí		
<u>ECDHE-RSA (forward secrecy)</u>							
<u>DH-DSS</u>	No	Sí	Sí	Sí	Sí		
<u>DHE-DSS (forward secrecy)</u>							
<u>ECDH-ECDSA</u>	No	No	Sí	Sí	Sí		
<u>ECDHE-ECDSA (forward secrecy)</u>							
<u>DH-ANON (inseguro)</u>	No	No	Sí	Sí	Sí		
<u>ECDH-ANON (inseguro)</u>	No	No	Sí	Sí	Sí		
<u>GOST R 34.10-94 / 34.10-2001¹⁴</u>	No	No	Sí	Sí	Sí		Propuesto en borradores RFC

Cifrado

Véase también: Cifrado por bloques

Seguridad del cifrado contra ataques conocidos

Cifrado			Versión del Protocolo						Estado
Tipo	Algoritmo	Fortaleza nominal (bits)	SSL 2.0	SSL 3.0 ^{note 1} ^{note 2} ^{note 3}	TLS 1.0 ^{note 1} ^{note 3}	TLS 1.1 ^{note 1}	TLS 1.2 ^{note 1}	TLS 1.3 ^{>15}	
Cifrado por bloques	AES GCM ¹⁶ ^{note 4}	256, 128	N/D	N/D	N/D	N/D	Seguro	Seguro	Definido para TLS 1.2 en RFCs
	AES CCM ¹⁷ ^{note 4}		N/D	N/D	N/D	N/D	Seguro	Seguro	
	AES CBC ^{note 5}		N/D	N/D	depende	depende	depende	N/D	
	Camellia GCM ¹⁸ ^{note 4}	256, 128	N/D	N/D	N/D	N/D	Seguro	N/D	
	Camellia CBC ¹⁹ ^{note 5}		N/D	N/D	depende	depende	depende	N/D	
	ARIA GCM	256, 128	N/D	N/D	N/D	N/D	Seguro	N/D	
	ARIA CBC		N/D	N/D	depende	depende	depende	N/D	
	SEED CBC ²⁰ ^{note 5}	128	N/D	N/D	depende de las mitigaciones	depende de las mitigaciones	depende de las mitigaciones	N/D	
	3DES EDE CBC ^{note 5} ^{note 6}	112 ^{note 7}	Inseguro	Inseguro	Inseguro	Inseguro	Inseguro	N/D	
	GOST 28147-89 CNT ¹⁴ ^{note 6}	256	N/D	N/D	Inseguro	Inseguro	Inseguro	N/D	Definido en IETF RFC 4357
	IDEA CBC ^{note 5} ^{note 6} ^{note 8}	128	Inseguro	Inseguro	Inseguro	Inseguro	N/D	N/D	Retirado de TLS 1.2
	DES CBC ^{note 5} ^{note 6} ^{note 8}	56	Inseguro	Inseguro	Inseguro	Inseguro	N/D	N/D	Prohibidas en TLS 1.1 y posteriores
		40 ^{note 9}	Inseguro	Inseguro	Inseguro	N/D	N/D	N/D	
	RC2 CBC ^{note 5} ^{note 6}	40 ^{note 9}	Inseguro	Inseguro	Inseguro	N/D	N/D	N/D	
Cifrador de flujo	ChaCha20+Poly1305 ²⁵ ^{note 4}	256	N/D	N/D	N/D	N/D	Seguro	Seguro	Definido para TLS 1.2 en RFCs
	RC4 ^{note 10}	128	Inseguro	Inseguro	Inseguro	Inseguro	Inseguro	N/D	Prohibido para todas las versiones de TLS en RFC 7465
		40	Inseguro	Inseguro	Inseguro	N/D	N/D	N/D	
Ninguno	Nulo ^{note 11}	–	N/D	Inseguro	Inseguro	Inseguro	Inseguro	N/D	Definido para TLS 1.2 en los RFCs

Notas

1. RFC 5746 debe estar implementado para solucionar una falla en la renegociación que de otra forma rompe este protocolo.
2. Si las librerías implementan las mejoras listadas en el RFC 5746, esto violaría la especificación SSL 3.0, la que la IETF no puede cambiar a diferencia de TLS. Afortunadamente, la mayoría de las librerías actuales implementan las mejoras y no se preocupan por la violación que esto causa.
3. el ataque BEAST rompe todos los bloques cifrados (CBC

ciphers) usados en SSL 3.0 y TLS 1.0 a menos de que sea mitigado por el cliente. A noviembre de 2013, Apple activó la mitigación por defecto solo para Safari 7 para Mac OS X 10.9, resultando en que tanto Safari for iOS, para Windows, y para Mac OS X 10.8 y anteriores todavía eran vulnerables al ataque BEAST en estas plataformas. - ver #Navegadores

4. AEAD cifrado (como GCM y CCM) puede ser usado solo en TLS 1.2.
5. El cifrado CBC puede ser atacado con el Ataque Trece

- con suerte si la librería no está escrita cuidadosamente para eliminar el timing de canales laterales.
6. El ataque Sweet32 rompe cifrados de bloque con bloques de tamaño de 64 bits.²¹
 7. 3DES provee solamente 108 o 112 bits de seguridad,²² lo cual es menos que el mínimo recomendado de 128 bits.²³
 8. IDEA y DES fueron removidas de TLS 1.2.²⁴
 9. El cifrado de 40 bits fue diseñado para operar con claves reducidas y así cumplir con regulaciones de EE.UU. respecto de la exportación de software criptográfico. Estas combinaciones están prohibidas en TLS 1.1 y posteriores.
 10. los ataques RC4 debilitan o rompen el RC4 usado en SSL/TLS
 11. solo Autenticación, sin encriptación.

Integridad de datos

Se utiliza código de autenticación de mensaje (MAC, por *message authentication code* en inglés) para asegurar la integridad de los datos. HMAC se usa para el modo CBC de cifrado de bloques y cifrado de *streams*. AEAD es usado para el cifrado autenticado tales como los modos GCM y CCM.

Integridad de Datos

Algoritmo	SSL 2.0	SSL 3.0	TLS 1.0	TLS 1.1	TLS 1.2	TLS 1.3	Estatus
HMAC-MD5	Sí	Sí	Sí	Sí	Sí		Definido para TLS 1.2 en RFCs
HMAC-SHA1	No	Sí	Sí	Sí	Sí		
HMAC-SHA256/384	No	No	No	No	Sí		
AEAD	No	No	No	No	Sí		
GOST 28147-89 IMIT ¹⁴	No	No	Sí	Sí	Sí		Propuesto en borradores de RFC
GOST R 34.11-94 ¹⁴	No	No	Sí	Sí	Sí		

Aplicaciones y adopción

SSL se ejecuta en una capa entre los protocolos de aplicación como HTTP, SMTP, NNTP y sobre el protocolo de transporte TCP, que forma parte de la familia de protocolos TCP/IP. Puede proporcionar seguridad a cualquier protocolo que use conexiones de confianza (tal como TCP).

Sitios web

Uno de los usos más importantes es junto a HTTP para formar HTTPS. HTTPS es usado para asegurar páginas World Wide Web para aplicaciones de comercio electrónico, utilizando certificados de clave pública para verificar la identidad de los extremos.

Soporte de protocolo de sitios web

Versión del Protocolo	Soporte en Sitios Web ²⁶	Seguridad ^{26 27}
SSL 2.0	16.6% (-2.8%)	Inseguro
SSL 3.0 ^{n 1}	60.6% (-37.4%)	Inseguro ²⁸
TLS 1.0	99.5% (±0.2%)	Depende del cifrado y de la mitigación de <u>BEAST</u> en el cliente ^{n 2 n 3 n 4 n 5}
TLS 1.1	45.4% (+3.4%)	Depende del cifrado ^{n 2 n 3 n 4 n 5}
TLS 1.2	48.1% (+3.8%)	Depende del cifrado ^{n 2 n 3 n 4 n 5}

Notas

1. La falla de renegociación insegura rompe este protocolo. Si las librerías implementan las mejoras listadas en el RFC 5746, esto violaría la especificación SSL 3.0, la que la IETF no puede cambiar a diferencia de TLS. Afortunadamente, la mayoría de las librerías actuales implementan las mejoras y no se preocupan por la violación que esto causa.
2. ver seguridad del cifrado web en la tabla de más abajo
3. varios ataques a RC4 debilitan o rompen el RC4 usado en SSL/TLS
4. el ataque CRIME significa que la compresión TLS no es segura, y que el ataque BREACH que requiere compresión HTTP puede ser usado para vencer la seguridad tanto de TLS y de SSL 3.0 que sea parcheado con el RFC 5746 cuando la compresión HTTP está habilitada.
5. RFC 5746 debe estar implementado en orden a resolver un defecto en la renegociación que de otra forma podría

romper el protocolo.

Navegadores

Todos los navegadores importantes soportan TLS:

Soporte de TLS en navegadores

Navegador	Plataforma	TLS 1.0	TLS 1.1	TLS 1.2
Chrome 0–22	Linux, Mac OS X, Windows (XP, Vista, 7, 8). ^{notas 1 29 30} ^{notas 2}	Sí	No	No
Chrome 22–29	Linux, Mac OS X, Windows (XP, Vista, 7, 8). ^{notas 1} ^{notas 2}	Sí	Sí	No
Chrome 30–	Linux, Mac OS X, Windows (XP, Vista, 7, 8). ^{notas 1} ^{notas 2}	Sí	Sí	Sí
Firefox 2–	Linux, Mac OS X, Windows (XP, Vista, 7, 8). ^{notas 2}	Sí ³¹	No ³²	No ³³
Firefox 27–	Linux, Mac OS X, Windows (XP, Vista, 7, 8). ^{notas 1} ^{notas 2}	Sí	Sí	Sí
IE 1–7	Mac OS X, Windows (XP, Vista, 7). ^{notas 3 34 35}	Sí	No	No
IE 8	Windows (XP, Vista). ^{notas 3}	Sí	No	No
IE 8–9	Windows 7. ^{notas 3}	Sí	Sí, deshabilitada por defecto	Sí, deshabilitada por defecto
IE 9	Windows Vista. ^{notas 3}	Sí	No	No
IE 10	Windows (7, 8). ^{notas 3}	Sí	Sí, deshabilitada por defecto	Sí, deshabilitada por defecto
IE 11	Windows (7, 8.1). ^{notas 3}	Sí	Sí	Sí
Opera 10–	Linux, Mac OS X, Windows. ^{notas 4 36}	Sí	Sí, deshabilitada por defecto	Sí, deshabilitada por defecto
Safari 5–6	Mac OS X, Windows (XP, Vista, 7). ^{notas 5} ^{notas 6 37 38}	Sí	No	No
Safari 7	Mac OS X 10.9. ^{notas 5}	Sí	Sí	Sí
Mobile Safari / UIWebView	iOS 5.0+. ^{notas 7 39 40 41}	Sí	Sí	Sí

Notas

- El navegador [Google Chrome](#) (y [Chromium](#)) soportan TLS 1.0 y TLS 1.1 desde la versión 22 (se agregó, luego se bajó desde la versión 21). TLS 1.2 no es compatible
- Utiliza la implementación TLS proporcionada por NSS. A noviembre de 2012, NSS soporta TLS 1.0 y 1.1, pero no 1.2. ^{[[actualizar](#)]}
- IE utiliza la implementación TLS del sistema operativo Microsoft Windows proporcionada por el proveedor soporte de seguridad SChannel. TLS 1.1 y 1.2 están deshabilitadas por defecto
- Hasta [Presto 2.2](#), que aparece en Opera 10, Opera añade soporte para TLS 1.2 después de soportar previamente 1.0 y 1.1.
- Safari utiliza la implementación del sistema operativo Mac OS X, Windows (XP, Vista, 7) con versión desconocida
- Safari 5 es la última versión disponible para Windows.
- Mobile Safari y software de terceros que utilizan el sistema [UIWebView](#) biblioteca utiliza el sistema operativo iOS aplicación, que admite TLS 1.2 Desde de iOS 5.0.

Bibliotecas

SSL y TLS han sido implementados ampliamente en varios proyectos de software abierto y libre. Los programadores pueden usar las librerías [PolarSSL](#), [CyaSSL](#), [OpenSSL](#), [MatrixSSL](#), [NSS](#) o [GnuTLS](#) para tener funcionalidad SSL/TLS.

- Microsoft Windows incluye una implementación de SSL y TLS como parte de su paquete Secure Channel.
- OS X incluye una implementación de SSL y TLS como parte de su paquete [Secure Transport](#).
- Los programadores de [Delphi](#) pueden usar una librería llamada [Indy](#).

- OpenSSL es una implementación libre. Tiene una [licencia BSD](#) con algunas extensiones.
- GnuTLS es una implementación libre, con licencia LGPL.
- Zodiac TLS/SSL: es una implementación para [Smalltalk](#) con [licencia MIT](#).
- [cryptlib](#) es librería criptográfica potable de software abierto (que incluye una implementación de SSL/TLS).
- JSSE: una implementación Java incluida en el [Java Runtime Environment](#) que soporta TLS 1.1 y 1.2 desde Java 7, aunque está por defecto deshabilitada para el cliente y habilitada en el servidor.
- MatrixSSL: una implementación con licencia dual.
- Network Security Services (NSS): es una librería open source validada para [FIPS 140](#).
- PolarSSL es una implementación SSL/TLS muy pequeña para dispositivos embebidos que está diseñada para uso fácil.
- CyaSSL es una librería SSL/TLS embebida con fuerte foco en velocidad y tamaño.

Un ensayo presentado en la conferencia ACM 2012 de seguridad de computadores y comunicaciones⁴² mostró que muchas aplicaciones utilizaban estas librerías incorrectamente, llevando a vulnerabilidad. Los autores hacían notar que "la causa principal de la mayoría de estas vulnerabilidad es el terrible diseño de las APIs para las librerías subyacentes. En vez de expresar propiedades de seguridad alto nivel para túneles de red tales como confidencialidad y autenticación, estas API exponen detalles de bajo nivel del protocolo SSL a los desarrolladores de aplicaciones. Como consecuencia, los desarrolladores frecuentemente usan las API de SSL incorrectamente, malinterpretando y malentendiendo los posibles parámetros, opciones, efectos colaterales y valores de retorno".

Otros usos

Otra aplicación con creciente uso de TLS es [SMTP](#). TLS es también el método estándar para proteger la señalización de aplicaciones con [Session Initiation Protocol](#) (SIP). TLS se puede utilizar para proveer autenticación y cifrado a la señalización asociada con [VoIP](#) y otras aplicaciones basadas en SIP.

Aunque un número creciente de productos clientes y servidores pueden proporcionar SSL de forma nativa, muchos aún no lo permiten. En estos casos, un usuario podría querer usar una aplicación SSL independiente como Stunnel para proporcionar cifrado. No obstante, el [Internet Engineering Task Force](#) recomendó en 1997 que los protocolos de aplicación ofrecieran una forma de actualizar a TLS a partir de una conexión sin cifrado (*plaintext*), en vez de usar un puerto diferente para cifrar las comunicaciones – esto evitaría el uso de envolturas (*wrappers*) como Stunnel.

SSL también puede ser usado para tunelizar una red completa y crear una [red privada virtual](#) (VPN), como en el caso de [OpenVPN](#).

Seguridad

SSL 2.0

SSL 2.0 tiene una variedad de fallas:⁴³

- Claves criptográficas idénticas se utilizan para la autenticación de mensajes y el cifrado.
- SSL 2.0 tiene una construcción [MAC](#) débil que utiliza la función *hash* MD5 con un prefijo secreto, por lo que es vulnerable a los ataques de extensión de longitud.
- SSL 2.0 no tiene ningún tipo de protección para el *handshake*, es decir, un ataque *man-in-the-middle* que se rebaje a este protocolo puede pasar desapercibido.
- SSL 2.0 utiliza el cierre de la conexión TCP para indicar el final de los datos. Esto significa que los ataques de truncamiento son posibles: el atacante simplemente forja un TCP FIN, dejando el receptor inconsciente de un fin ilegítimo de mensaje de datos (SSL 3.0 soluciona este problema al tener una alerta de cierre explícita).
- SSL 2.0 asume un solo servicio y un certificado de dominio fijo, que choca con una función estándar de *hosting* virtual en los servidores Web. Esto significa que la mayoría de los sitios web están prácticamente afectados por el uso de SSL.

SSL 2.0 está desactivado por defecto, a partir de: [Internet Explorer 7](#),⁴⁴ [Mozilla Firefox 2](#),⁴⁵ [Opera 9.5](#),⁴⁶ y Safari. Después de que se envía un "ClientHello" TLS, si Mozilla Firefox comprueba que el servidor no puede completar el *handshake*, intentará volver a caer a la utilización de SSL 3.0 con un SSL 3.0 "ClientHello" en formato SSL 2.0 para maximizar la probabilidad de éxito del *handshake* con los servidores más antiguos.⁴⁷ Permitir SSL 2.0 (y sistemas de cifrado débiles de 40 y 56 bits), ha sido completamente eliminado de Opera desde la versión 10⁴⁸

⁴⁹

SSL 3.0

SSL 3.0 mejoró SSL 2.0 mediante la adición de cifrado [SHA-1](#) y soporte para autenticación de certificados.

Desde el punto de vista de seguridad, SSL 3.0 debería considerarse menos deseable que TLS 1.0. Las suites de cifrado de SSL 3.0 tienen un proceso de derivación de claves débiles, la mitad de la llave maestra que se establece es totalmente dependiente de la función *hash* [MD5](#), que

no es resistente a los choques y, por lo tanto, no es considerado seguro. Bajo TLS 1.0, la llave maestra que se establece depende tanto MD5 y SHA-1 por lo que su proceso de derivación no está actualmente considerado débil. Es por esta razón que las implementaciones SSL 3.0 no pueden ser validados bajo FIPS 140-2.⁵⁰

Hay algunos ataques contra la implementación en lugar del propio protocolo:⁵¹ En las implementaciones anteriores, algunas entidades emisoras⁵² no establecieron explícitamente `basicConstraintsCA=False` para los *nodos hoja*. Como resultado, estos *nodos hoja* podían firmar certificados piratas. Además, algunos programas de (incluyendo IE6 y *Konqueror*) no comprobó este campo para nada. Esto puede ser explotado en ataques *man-in-the-middle* a todas las posibles conexiones SSL. Algunas implementaciones (incluyendo versiones anteriores de la API de cifrado de Microsoft, Network Security Services y GnuTLS) dejan de leer los caracteres que siguen al carácter nulo en el campo del nombre del certificado, lo que puede ser explotado para engañar al cliente en la lectura del certificado como si fuera originado en el sitio auténtico. (Por ejemplo, PayPal.com\0.badguy.com sería confundido como proveniente del sitio PayPal.com en lugar de badguy.com). Los navegadores implementaron mecanismos de degradación del protocolo a una versión anterior en SSL/TLS por razones de compatibilidad. La protección ofrecida por los protocolos SSL/TLS contra un downgrade a una versión anterior de un ataque *man-in-the-middle* activo puede ser inutilizados por tales mecanismos.⁵³

TLS

TLS tiene una variedad de medidas de seguridad:

- Protección contra una degradación del protocolo a una versión anterior (menos segura) o un conjunto de cifrado más débil.
- Numeración de los registros de aplicación posteriores con un número de secuencia y el uso de este número de secuencia en los códigos de autenticación de mensajes (MAC).
- Usando un resumen de mensaje mejorado con una clave (por lo que sólo una llave-sostenedor puede comprobar el MAC). La construcción HMAC utilizado por la mayoría de las suites de cifrado TLS se especifica en el RFC 2104 (SSL 3.0 utiliza un MAC basado en *hash* diferente).
- El mensaje que finaliza el protocolo de enlace ("Finalizar") envía un *hash* de todos los mensajes intercambiados *handshake* vistos por ambas partes.
- La función pseudoaleatoria divide los datos de entrada en un medio y procesa cada uno con un algoritmo de *hash* diferente (MD5 y SHA-1), luego les hace OR exclusivo juntos para crear el MAC. Esto proporciona protección incluso si uno de estos algoritmos resulta ser vulnerable.

Ataques contra SSL/TLS

Los ataques más significativos se mencionan más abajo:

Ataque de renegociación

Una vulnerabilidad del procedimiento en el cual la renegociación fue descubierto en agosto de 2009, que puede conducir a ataques de inyección de texto plano contra SSL 3.0 y todas las versiones actuales de TLS. Por ejemplo, permite a un atacante que puede secuestrar una conexión https para empalmar sus propias peticiones en el inicio de la conversación que el cliente tiene con el servidor web. El atacante no puede realmente descifrar la comunicación cliente-servidor, por lo que es diferente de un típico ataque *man-in-the-middle*. Una solución a corto plazo es que los servidores de Internet dejen de permitir la renegociación, que normalmente no requerirá otros cambios a menos que se utilice la autenticación de certificados de cliente. Para corregir la vulnerabilidad, una extensión de la indicación de renegociación fue propuesta para TLS. Se requerirá que el cliente y el servidor incluyan y verifiquen información acerca de los *handshake* anteriores en cualquier renegociación de *handshake*.⁵⁴ Esta extensión se ha convertido en una norma propuesta y se le ha asignado el número de RFC 5746. El RFC ha sido implementado por varias bibliotecas.^{55 56 57}

Ataques de reversión de versiones

Hay modificaciones a los protocolos originales, como False Start⁵⁸ (aprobada y habilitada por Google Chrome⁵⁹) o Snap Start, en las que se ha reportado que han introducido limitaciones a los ataques de reversión de versiones para TLS⁶⁰ o para permitir que las modificaciones de la lista de conjunto de cifrado enviada por el cliente al servidor (un atacante puede ser capaz de influir en la selección de la suite de cifrado en un intento de rebajar la intensidad de juego de cifrado, ya sea para usar un algoritmo de cifrado simétrico más débil o de un intercambio de clave más débil⁶¹). Se ha demostrado en la conferencia sobre seguridad informática y de comunicaciones de la Association for Computing Machinery (ACM) que la extensión False Start está en riesgo bajo ciertas circunstancias, lo que podría permitir a un atacante recuperar las claves de cifrado en línea y acceder a los datos cifrados.⁶²

Ataque BEAST

El 23 de septiembre de 2011, los investigadores Thai Duong y Juliano Rizzo demostraron una "prueba de concepto" llamada BEAST ("Browser Exploit Against SSL/TLS") usando un applet Java para violar restricciones de políticas de mismo origen, por una vulnerabilidad de CBC ampliamente conocida de TLS 1.0.^{51 52} Exploits prácticos de esta vulnerabilidad no se conocían, la cual fue descubierta originalmente por Phillip Rogaway⁶³ en 2002.

Mozilla actualizó las versiones de desarrollo de sus librerías NSS para mitigar ataques de tipo BEAST. NSS es utilizado por Mozilla Firefox y por Google Chrome para su implementación de SSL. Algunos servidores web que tienen una implementación quebrada de la especificación SSL puede que dejen de funcionar como resultado de esto.⁶⁴

Microsoft emitió el boletín de seguridad MS12-006 el 12 de enero de 2012, que corrigió la vulnerabilidad BEAST al cambiar la forma en que el componente de Windows Secure Channel (SChannel) transmite los paquetes cifrados.⁶⁵ Por su parte, Apple habilitó por defecto la protección contra BEAST en la versión OS X 10.9 Mavericks.⁶⁶

El ataque BEAST también se puede prevenir eliminando todos los cifrados CBC de la lista de cifrados permitidos, dejando solamente el cifrado RC4, que es ampliamente soportado por la mayoría de los sitios web.^{67 68} Los usuarios de Windows 7 y de Windows Server 2008 R2 pueden permitir el uso de TLS 1.1 y 1.2, pero esta contramedida fallará si no es soportado también por el otro extremo de la conexión, y caerá a TLS 1.0.

Ataques CRIME y BREACH

Los autores del ataque BEAST también son los creadores del ataque CRIME, que usa compresión de datos para adivinar.^{69 70} Cuando se utiliza para recuperar el contenido de la cookie de autenticación secreta, permite a un atacante realizar un secuestro de sesión en una sesión web autenticada.

Ataques de relleno

Las versiones anteriores de TLS eran vulnerables frente al ataque de relleno de oráculo descubierto en 2002. Una nueva variante, llamada Ataque Trece con suerte, fue publicada en 2013. Hasta febrero de 2013, los implementadores de TLS estaban todavía trabajando en el desarrollo de soluciones para la protección contra esta forma de ataque.

Una solución definitiva fue lanzada como la extensión Encrypt-then-MAC para TLS lanzado como RFC 7366.⁷¹

Ataque POODLE

El 14 de octubre de 2014, investigadores de Google publicaron una vulnerabilidad en el diseño de SSL 3.0, lo que hace que el modo CBC de operación con SSL 3.0 sea vulnerable al ataque de relleno (CVE-2014-3566 (<https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-3566>)). Ellos llamaron a este ataque POODLE (en inglés, Padding Oracle On Downgraded Legacy Encryption o Relleno de oráculo en Degradación a Cifrado Obsoleto). En promedio, los atacantes solo necesitan hacer 256 peticiones SSL 3.0 para revelar un byte de mensaje cifrado.^{28 72}

Aunque esta vulnerabilidad solo existe en SSL 3.0 y la mayoría de los clientes y servidores admite TLS 1.0 y superiores, todos los principales navegadores rebajan voluntariamente a SSL 3.0 si los *handshake* con las nuevas versiones de TLS fallan a menos que proporcionan la opción para un usuario o administrador para deshabilitar SSL 3.0 y el usuario o el administrador lo haga. Por lo tanto, el hombre-en-el-medio primero debe llevar a cabo un ataque de *rollback* y luego aprovechar esta vulnerabilidad.^{28 72}

En general, la degradación de la seguridad elegante por el bien de la interoperabilidad es difícil llevar a cabo de una manera que no pueda ser explotada. Este es un reto especialmente en los dominios donde la fragmentación es alta.⁷³

Ataque RC4

A pesar de ataques existentes sobre RC4 que lo rompen, las suites de cifrado basadas en RC4 en SSL y TLS fueron consideradas seguros en un momento debido a la forma en que el sistema de cifrado se utilizaba en estos protocolos derrotaba a los ataques que rompían RC4, hasta que nuevos ataques divulgados en marzo de 2013 permitían que RC4 en TLS fuera quebrado completamente. En 2011 se recomendaba usar la suite RC4 como una solución alternativa para el ataque BEAST.⁷⁴ En 2013 una vulnerabilidad fue descubierta en RC4 sugiriendo que no era una buena solución para BEAST.²⁷ Un caso de un ataque fue propuesto por Alfaridan, Bernstein, Paterson, Poettering y Schuldt que utilizaba nuevos sesgos estadísticos descubiertos en la tabla de clave RC4⁷⁵ para recuperar partes del texto en claro con un gran número de cifrados TLS.^{76 77} Un ataque de sesgo de doble byte en RC4 en TLS y SSL que requiere 13×220 cifrados para romper RC4 se dio a conocer el 8 de julio de 2013, y fue descrito como "viable" en la presentación de acompañamiento en el 22º Simposio USENIX de Seguridad el 15 de agosto de 2013.^{78 79}

Sin embargo, muchos navegadores modernos han sido diseñados para derrotar los ataques BEAST (excepto Safari para Mac OS X 10.7 o versiones anteriores, para iOS 6 o anterior, y para Windows; ver navegadores). Como resultado, RC4 ya no es la mejor opción para TLS 1.0. Los sistemas de cifrado CBC que se vieron afectados por el ataque BEAST en el pasado se están convirtiendo en una opción más popular para la protección.²³

Microsoft recomienda deshabilitar RC4 cuando sea posible.⁸⁰

Ataque de truncamiento

Un ataque de truncamiento TLS bloquea las peticiones de desconexión de la cuenta de la víctima para que el usuario sin saberlo, permanezca conectado a un servicio web. Cuando se envía la solicitud de fin de sesión, el atacante inyecta un mensaje TCP FIN no cifrado (no hay más datos del remitente) para cerrar la conexión. El servidor, por tanto, no recibe la solicitud de cierre de sesión y no se da cuenta de la terminación

anormal.⁸¹

Publicado en julio de 2013,⁸² el ataque provoca servicios web como Gmail y Hotmail que muestren una página que informa al usuario de que han salido correctamente del servicio, al tiempo que garantiza que el navegador del usuario mantiene la autorización con el servicio, lo que permite a un atacante tener el acceso para tomar el control de la cuenta que ha iniciado sesión en el usuario. El ataque no se basa en la instalación de *malware* en el ordenador de la víctima; los atacantes solo necesitan ponerse entre la víctima y el servidor web (por ejemplo, mediante la creación de un punto de acceso inalámbrico rebelde).⁸¹ Esta vulnerabilidad también requiere acceso a la computadora de la víctima.

Fallo Heartbleed

El fallo Heartbleed es una grave vulnerabilidad en la popular librería de software criptográfica OpenSSL, que afecta a las versiones 1.0.1 a 1.0.1f. Esta debilidad permite el robo de la información protegida, en condiciones normales, por el cifrado SSL / TLS que se utiliza para asegurar las cargas de datos. SSL / TLS proporciona seguridad de las comunicaciones y la privacidad a través de Internet para aplicaciones como web, correo electrónico, mensajería instantánea (IM) y algunas redes privadas virtuales (VPN).⁸³

El fallo Heartbleed permite a cualquier persona en Internet leer la memoria de los sistemas protegidos por las versiones vulnerables del software OpenSSL. Esto compromete las claves secretas utilizadas para identificar los proveedores de servicios y para cifrar el tráfico, los nombres y las contraseñas de los usuarios y el contenido real. Esto permite a los atacantes espiar las comunicaciones, robar datos directamente de los servicios y de los usuarios y suplantar servicios y a los usuarios.⁸⁴

Estudio de sitios web

A diciembre de 2014, Trustworthy Internet Movement estima que la proporción de sitios web que son vulnerable a ataques TLS.²⁶

Estudio de las vulnerabilidades de TLS de los sitios web más populares

Ataques	Seguridad				
		Inseguro	Depende	Seguro	Otro
Ataque de Renegociación	—	3.7% (−0.2%) soporta renegociación insegura	1.2% (±0.0%) soporta ambos	90.0% (+0.5%) soporta renegociación segura	5.1% (−0.3%) sin soporte
Ataque RC4	1.3% (−0.1%) permite sólo cifrados RC4	26.6% (−0.7%) permite cifrados RC4 usado con navegadores modernos	53.2% (−0.7%) soporta algunos cifrados RC4	20.2% (+1.4%) sin soporte	—
Ataque BEAST (mitigado en el lado del cliente con navegadores modernos)	—	77.4% (+0.6%) vulnerable	—	—	—
Ataque CRIME	—	7.2% (−0.4%) vulnerable	—	—	—
Heartbleed	—	0.3% (−0.1%) vulnerable	—	—	—
Ataque de inyección ChangeCipherSpec	—	3.3% (−0.4%) vulnerable y explotable	15.5% (−1.1%) vulnerable, no explotable	80.1% (+1.3%) no vulnerable	1.0% (+0.1%) desconocido
POODLE contra TLS (POODLE Original contra SSL 3.0 no está incluido)	—	10.1% vulnerable y explotable	—	88.4% no vulnerable	1.6% desconocido

Forward secrecy

Forward secrecy es una propiedad de los sistemas criptográficos que garantiza que una clave de sesión derivada de un conjunto de claves públicas y privadas no se verán comprometidas si una de las claves privadas se ve comprometida en el futuro.⁸⁵ Sin forward secrecy, si la clave privada del servidor fuera conocida, no sólo se verán comprometidas todas las sesiones cifradas-TLS futuras mediante ese certificado del servidor, sino también las sesiones anteriores que lo utilizaban (siempre por supuesto que estas sesiones pasadas fueran interceptadas y almacenadas en el momento de la transmisión).⁸⁶ Una implementación de TLS puede proporcionar forward secrecy al exigir el uso de intercambio de claves *Diffie-Hellman* efímeras para establecer claves de sesión, y algunas implementaciones TLS notables lo hacen exclusivamente: por ejemplo, Gmail y otros servicios de Google que utilizan HTTPS *OpenSSL*.⁸⁷ Sin embargo, muchos clientes y servidores que soportan TLS (incluidos los navegadores y servidores web) no están configurados para aplicar esas restricciones.^{88 89} En la práctica, a menos que un servicio web utilice Diffie-Hellman para implementar forward secrecy, todo el tráfico web cifrado hacia y desde ese servicio puede ser descifrado por un tercero si obtiene la clave maestra del servidor (privado); por ejemplo, por medio de una orden judicial.⁹⁰

Incluso cuando se implementa el intercambio de claves Diffie-Hellman, los mecanismos de gestión de sesión en el servidor pueden afectar el

forward secrecy. El uso de tickets de sesión TLS (una extensión TLS) hace que la sesión se deba proteger por AES128-CBC-SHA256 independientemente de cualquier otro parámetro TLS negociados, incluyendo a los conjuntos de cifrado de forward secrecy, y las llaves de los tickets de sesión de larga duración TLS estropean el intento de implementar forward secrecy.^{91 92 93}

Desde finales de 2011, Google ha proporcionado forward secrecy con TLS por omisión para los usuarios de su servicio de [Gmail](#), junto con [Google Docs](#) y búsqueda cifrada, entre otros servicios.⁹⁴ Desde noviembre de 2013, [Twitter](#) ha proporcionado forward secrecy con TLS para los usuarios de su servicio.⁹⁵ A diciembre de 2014, el 20,0% de los sitios web habilitados para TLS están configurados para utilizar conjuntos de cifrado que proporcionan forward secrecy a los navegadores web.²⁶

Evitar Triple-DES CBC

Algunos expertos recomiendan evitar Triple-DES CBC. Debido a que RC4 y Triple-DES son los últimos cifrados soportados en las bibliotecas SSL/TLS de [Windows XP](#), hace que sea difícil soportar SSL para programas que utilicen esta biblioteca en Windows XP. Por ejemplo en el caso de Internet Explorer para Windows XP.²³

Hacer frente a los ataques MITM

Fijación de Certificados

Una forma de detectar y bloquear muchos tipos de ataques MITM es "fijar el certificado", a veces llamado "fijación SSL".⁹⁶

Un cliente que hace fijación de certificado agrega un paso adicional para el protocolo SSL o protocolo TLS habitual: Después de obtener el certificado del servidor en la forma estándar, el cliente comprueba el certificado del servidor con los datos de confianza para validación. Normalmente los datos de validación de confianza se incluye con la aplicación, en la forma de una copia de confianza de dicho certificado, o un *hash* de confianza o huella digital del certificado o la clave pública del certificado. Por ejemplo, el Chromium y Google Chrome incluyen datos de validación para el certificado `*.google.com` que detecta [certificados fraudulentos](#) en 2011. Desde entonces, Mozilla ha introducido fijación de certificados públicos en su navegador [Firefox](#).⁹⁷

En otros sistemas que el cliente espera que la primera vez que obtenga el certificado de un servidor es de confianza y la almacena; durante las sesiones posteriores con ese servidor, el cliente comprueba el certificado del servidor contra el certificado almacenado para protegerse de los ataques MITM posteriores.

Proyecto Perspectivas

El Proyecto Perspectivas⁹⁸ opera notarios de red que los clientes pueden usar para detectar si el certificado de un sitio ha cambiado. Por su naturaleza, los ataques man-in-the-middle colocan al atacante entre el destino y un solo objetivo específico. Como tal, Perspectivas advertirían al objetivo de que el certificado entregado al navegador web no coincide con el certificado visto desde otras perspectivas - las perspectivas de otros usuarios en diferentes momentos y lugares. El uso de notarios de red desde una multitud de perspectivas hace posible que un objetivo detecte un ataque incluso si un certificado parece ser completamente válido.

Detalles del Protocolo

El protocolo TLS intercambia *registros*- los que encapsulan los datos que se intercambian en un formato específico (ver más abajo). Cada registro puede ser comprimido, cifrado y empaquetado con un código de MAC (Código de Autenticación del Mensaje), todo dependiendo del estado de la conexión. Cada registro tiene un *campo de tipo de contenido* que designa el tipo de datos encapsulados, un campo de longitud y un campo de versión TLS. Los datos encapsulados pueden ser mensajes de control o de procedimiento de la propia TLS, o simplemente los datos de las aplicaciones que necesitan ser transferidos por TLS. Las especificaciones (suite de cifrado, claves, etc.) necesarios para el intercambio de datos de la aplicación por TLS, se han acordado en el "handshake TLS" entre el cliente que solicita los datos y el servidor que responde a las solicitudes. Por tanto, el protocolo define tanto la estructura de cargas útiles transferidos en TLS y el procedimiento para establecer y supervisar la transferencia.

Handshake TLS

Cuando se inicia la conexión, el registro encapsula un protocolo de "control"- el protocolo de mensajería de *handshake* (contenido de tipo 22). Este protocolo se utiliza para el intercambio de toda la información requerida por las dos partes para el intercambio de los datos de las aplicaciones reales por TLS. En él se definen los mensajes de formato o que contengan esta información y el orden de su intercambio. Estos pueden variar en función de las demandas del cliente y del servidor, es decir, existen varios procedimientos posibles para establecer la conexión. Este intercambio inicial resulta en una conexión exitosa TLS (ambas partes listas para transferir datos de la aplicación con TLS) o un mensaje de alerta (como se especifica más adelante).

Handshake básico

A continuación, un ejemplo simple de conexión, que ilustra un *handshake* en el que el servidor (pero no el cliente) es autenticado por su certificado:

1. Fase de negociación:

- Un cliente envía un mensaje **ClientHello** especificando la versión más alta de protocolo TLS que soporta, un número al azar, una lista de conjuntos de cifrado sugeridos y métodos de compresión sugeridos. Si el cliente está intentando realizar un *handshake* reanudado, puede enviar un ID de sesión.
 - El servidor responde con un mensaje **ServerHello**, que contiene la versión del protocolo elegido, un número aleatorio, CipherSuite y método de compresión de las opciones ofrecidas por el cliente. Para confirmar o permitir *handshake* reanudado el servidor puede enviar un ID de sesión. La versión del protocolo elegido debe ser el más alto que tanto el soporte de cliente y servidor. Por ejemplo, si el cliente es compatible con la versión 1.1 y el servidor es compatible con la versión 1.2, la versión 1.1 se debe seleccionar; 1.0 no se debe seleccionar.
 - El servidor envía su mensaje de certificado (dependiendo de la suite de cifrado seleccionado, esto puede ser omitido por el servidor).⁹⁹
 - El servidor envía su mensaje **ServerKeyExchange** (en función del conjunto de cifrado seleccionado, esto puede ser omitido por el servidor). Este mensaje se envía a todos los conjuntos de cifrado DHE y DH_anon.³
 - El servidor envía un mensaje **ServerHelloDone**, lo que indica que terminó con la negociación del handshake.
 - El cliente responde con un mensaje **ClientKeyExchange**, que puede contener una PreMasterSecret, la clave pública, o nada. (Una vez más, esto depende de la cifra seleccionada.) Esta PreMasterSecret se cifra utilizando la clave pública del certificado del servidor.
 - El cliente y el servidor a continuación utilizan los números aleatorios y PreMasterSecret para calcular un secreto común, llamado el "secreto principal" (*master secret*). Todos los demás datos clave para esta conexión se deriva de este secreto principal (y los valores aleatorios generados tanto por cliente y por servidor), que se pasan a través de una función pseudoaleatoria cuidadosamente diseñado.
2. El cliente ahora envía un registro **ChangeCipherSpec**, esencialmente diciendo al servidor, "Todo lo que yo te diga de ahora en adelante será autenticado (y cifrada si los parámetros de cifrado estaban presentes en el certificado del servidor)." El ChangeCipherSpec es en sí mismo un protocolo a nivel de registro con el tipo de contenido 20.
- Por último, el cliente envía un mensaje autenticado y cifrado de *Finished* (terminado), que contiene un *hash* y MAC sobre los mensajes de *handshake* anteriores.
 - El servidor intentará descifrar el mensaje **Finished** del cliente y verificar el *hash* y MAC. Si el descifrado o verificación falla, el *handshake* se considera que ha fracasado y la conexión debe ser derribada.
3. Por último, el servidor envía un **ChangeCipherSpec**, diciéndole al cliente, "Todo lo que yo te diga de ahora en adelante será autenticado (y cifrado, si el cifrado se negoció)."
- El servidor envía su mensaje **Finished** autenticado y cifrado.
 - El cliente realiza el mismo descifrado y verificación.
4. Fase de aplicación: en este punto, el "handshake" está completado y el protocolo de aplicación está activada, con el tipo de contenido de 23. Los mensajes de aplicación intercambiados entre el cliente y el servidor también serán autenticados y opcionalmente cifrados exactamente igual que en su mensaje final. De lo contrario, el tipo de contenido contestará 25 y el cliente no va será autenticado.

Handshake TLS autenticado por el cliente

El siguiente ejemplo completo muestra un cliente siendo autenticado (además del servidor como el de más arriba) a través de TLS mediante certificados intercambiados entre ambos interlocutores.

1. Fase de Negociación:

- Un cliente envía un mensaje **ClientHello** especificando la versión más alta de protocolo TLS que soporta, un número al azar, una lista de conjuntos de cifrado sugeridos y métodos de compresión.
- El servidor responde con un mensaje **ServerHello**, que contiene la versión elegida del protocolo, un número al azar, una suite de cifrado y el método de compresión de las opciones ofrecidas por el cliente. El servidor también puede enviar un identificador de sesión como parte del mensaje para realizar un *handshake* reanudado.
- El servidor envía su mensaje de certificados (dependiendo de la suite de cifrado seleccionado, esto puede ser omitido por el servidor).⁹⁹
- El servidor envía su mensaje **ServerKeyExchange** (en función del conjunto de cifrado seleccionado, esto puede ser omitido por el servidor). Este mensaje se envía a todos los conjuntos de cifrado DHE y DH_anon.³
- El servidor solicita un certificado desde el cliente, de modo que la conexión pueda ser mutuamente autenticada, utilizando un mensaje **CertificateRequest**.
- El servidor envía un mensaje **ServerHelloDone**, lo que indica que terminó con la negociación handshake.
- El cliente responde con un mensaje de certificado, que contiene el certificado del cliente.

- El cliente envía un mensaje **ClientKeyExchange**, que puede contener un *PreMasterSecret*, clave pública, o nada. (Una vez más, esto depende del cifrado seleccionado.) Esta *PreMasterSecret* se cifra utilizando la clave pública del certificado del servidor.
 - El cliente envía un mensaje **CertificateVerify**, que es una firma en los mensajes de reconocimiento anteriores utilizando la clave privada del certificado del cliente. Esta firma puede ser verificada utilizando la clave pública del certificado del cliente. Esto permite al servidor saber que el cliente tiene acceso a la clave privada del certificado y por lo tanto posee el certificado.
 - El cliente y el servidor a continuación, utilizan los números aleatorios y *PreMasterSecret* para calcular un secreto común, llamado el "secreto principal". Todos los demás datos clave para esta conexión se derivan de este secreto maestro (y los valores aleatorios cliente-y generados por el servidor), que se pasa a través de una función pseudoaleatoria cuidadosamente diseñada.
2. El cliente ahora envía un registro **ChangeCipherSpec**, esencialmente diciendo al servidor, "Todo lo que yo te diga de ahora en adelante será autenticado (y cifrada si el cifrado se negoció)." El **ChangeCipherSpec** es en sí mismo un protocolo a nivel de registro y es tipo 20 y no 22.
- Por último, el cliente envía un mensaje cifrado de **Finished**, que contiene un *hash* y MAC sobre los mensajes de *handshake* anteriores.
 - El servidor intentará descifrar el mensaje final del cliente y verificar el *hash* y MAC. Si el descifrado o verificación falla, el *handshake* se considera que ha fracasado y la conexión debe ser derribada.
3. Por último, el servidor envía un **ChangeCipherSpec**, diciéndole al cliente, "Todo lo que yo te diga de ahora en adelante será autenticado (y cifrada si el cifrado se negoció)."
- El servidor envía su propio mensaje cifrado de **Finished**.
 - El cliente realiza el mismo descifrado y verificación.
4. Fase de aplicación: en este punto, el "handshake" está completado y el protocolo de aplicación está activado, con el tipo de contenido 23. Los mensajes de la aplicación intercambiados entre el cliente y el servidor también serán cifrados exactamente igual que en su mensaje **Finished**.

Handshake reanudado

Las operaciones de clave pública (por ejemplo, RSA) son relativamente costosas en términos de cálculo computacional. TLS proporciona un acceso directo seguro en el mecanismo de *handshake* para evitar estas operaciones: reanudar sesiones. Las sesiones reanudadas se implementan utilizando los identificadores (IDs) de sesión o *tickets* de sesión.

Aparte de la ventaja de rendimiento, reanudando las sesiones también se puede utilizar para inicio de sesión único, ya que se garantiza que tanto la sesión original, así como cualquier sesiones reanudada se originan desde el mismo cliente. Esto es de particular importancia para el FTP sobre el protocolo TLS/SSL, ya que de lo contrario podría sufrir de un ataque man-in-the-middle en el que un atacante podría interceptar el contenido de las conexiones de datos secundarias.¹⁰⁰

IDs de sesión

En un *handshake* normal *completo*, el servidor envía un *ID de sesión* como parte del mensaje **ServerHello**. El cliente asocia este *ID de sesión* con la dirección IP del servidor y el puerto TCP, para que cuando el cliente se conecte de nuevo a ese servidor, puede utilizar el *ID de sesión* para acortar el *handshake*. En el servidor, el *ID de sesión* se asigna a los parámetros criptográficos negociados anteriormente, específicamente el "secreto maestro". Ambas partes deben tener el mismo "secreto maestro" o el *handshake* reanudado fallará (esto evita que un espía utilice un *ID de sesión*). Los datos aleatorios en los mensajes **ClientHello** y **ServerHello** prácticamente garantizan que las claves de conexión generadas serán diferentes de la conexión anterior. En las RFC, este tipo de *handshake* se llama un protocolo de enlace abreviado. También se describe en la literatura como un reinicio de *handshake*.

1. Fase de Negociación:

- Un cliente envía un mensaje **ClientHello** especificando la versión más alta de protocolo TLS que soporta, un número al azar, una lista de conjuntos de cifrado sugeridos y métodos de compresión. Incluye en el mensaje el ID de sesión de la conexión TLS previa.
 - El servidor responde con un mensaje **ServerHello**, que contiene la versión elegida del protocolo, un número al azar, una suite de cifrado y el método de compresión de las opciones ofrecidas por el cliente. Si el servidor reconoce el *ID de sesión*, responde con la misma *ID de sesión*. El cliente usa esto para reconocer que se está llevando a cabo una sesión reanudada. Si el servidor no reconoce el *ID de sesión* enviado por el cliente, responde con un valor diferente para la *ID de sesión*, lo cual le dice al cliente que no se llevará a cabo una reanudación de sesión. En este punto, tanto el cliente como el servidor tienen el "secreto maestro" y datos aleatorios para generar la clave usada para esta conexión.
2. El servidor envía un **ChangeCipherSpec**, diciéndole al cliente, "Todo lo que yo te diga de ahora en adelante será

autenticado (y cifrada si el cifrado se negoció)."

- Por último, el servidor envía un mensaje cifrado de **Finished**, que contiene un *hash* y MAC sobre los mensajes de *handshake* anteriores.
 - El cliente realiza el mismo descifrado y verificación.
3. El cliente ahora envía un registro **ChangeCipherSpec**, esencialmente diciendo al servidor, "Todo lo que yo te diga de ahora en adelante será autenticado (y cifrada si el cifrado se negoció)."
- El cliente envía su propio mensaje cifrado de **Finished**.
 - El servidor intentará descifrar el mensaje final del cliente y verificar el *hash* y MAC.
4. Fase de aplicación: en este punto, el "handshake" está completado y el protocolo de aplicación está activado, con el tipo de contenido 23. Los mensajes de la aplicación intercambiados entre el cliente y el servidor también serán cifrados exactamente igual que en su mensaje **Finished**.

Tickets de sesión

El [RFC 5077](#) extiende TLS a través de la utilización de tickets de sesión, en lugar de los IDs de sesión. Define una forma de reanudar una sesión TLS sin necesidad de almacenar el estado específico de la sesión en el servidor TLS.

Al utilizar tickets de sesión, el servidor TLS almacena su estado específico de la sesión en un ticket de sesión y envía el ticket de sesión al cliente TLS para ser almacenado. El cliente se reanuda una sesión TLS mediante el envío del ticket de sesión al servidor, y el servidor reanuda la sesión TLS en el estado específico de la sesión en el ticket. El ticket de sesión está cifrado y autenticada por el servidor, y el servidor verifica su validez antes de utilizar su contenido.

Una debilidad particular de este método con [OpenSSL](#) es que siempre limita el cifrado y la autenticación de seguridad del ticket de sesión TLS transmitido a AES128-CBC-SHA256, no importa qué otros parámetros TLS sean negociados para la sesión actual TLS.^{[92](#)} Esto significa que la información de estado (el ticket de sesión TLS) no está tan bien protegido como la sesión TLS misma. De particular preocupación es el almacenamiento de OpenSSL de las claves en un contexto de aplicación (SSL_CTX), es decir, que se mantiene durante la duración de la aplicación, y no permite el reingreso de información de los tickets de sesión AES128-CBC-SHA256 TLS sin reiniciar el contexto a nivel de aplicación OpenSSL (lo que es raro, propenso a errores y a menudo requiere intervención administrativa manual).^{[93](#)} ^{[91](#)}

Registro TLS

Este es el formato general para todos los registros TLS.

+	Byte +0	Byte +1	Byte +2	Byte +3
Byte 0	Tipo de Contenido			
Bytes 1..4	Versión		Longitud	
	(Major)	(Minor)	(bits 15..8)	(bits 7..0)
Bytes 5..(m-1)	Mensaje(s) del Protocolo			
Bytes m..(p-1)	<u>MAC</u> (opcional)			
Bytes p..(q-1)	Relleno (cifrados de bloque solamente)			

Tipo de Contenido

Este campo identifica el Tipo de Capa de Registro del Protocolo contenido en este registro.

Tipos de Contenido

Hex	Dec	Tipo
0x14	20	ChangeCipherSpec
0x15	21	Alerta
0x16	22	Handshake
0x17	23	Aplicación
0x18	24	Heartbeat

Versión

Este campo identifica la versión mayor y menor de TLS para el mensaje contenido. Para un mensaje ClientHello, esto no tiene por qué ser la versión *más alta* admitida por el cliente.

Versiones

Versión Mayor	Versión Menor	Tipo de Versión
3	0	SSL 3.0
3	1	TLS 1.0
3	2	TLS 1.1
3	3	TLS 1.2

Longitud

La longitud del mensaje del Protocolo (s), MAC y Relleno, que no exceda de 214 bytes (16 KiB).

Protocolo del Mensaje

Uno o más mensajes identificados por el campo Protocolo. Tenga en cuenta que este campo puede ser cifrado dependiendo del estado de la conexión.

MAC y Relleno

Un código de autenticación de mensaje calculado sobre el mensaje del Protocolo, con el material clave adicional incluido. Tenga en cuenta que este campo puede ser codificado, o no incluidos en su totalidad, dependiendo del estado de la conexión.

No MAC o relleno pueden estar presentes al final de los registros TLS antes de todos los algoritmos de cifrado y parámetros han sido negociados y handshaked y luego confirmado por el envío de un registro CipherStateChange (ver más abajo) para la señalización de estos parámetros tendrán efecto en todos los registros adicionales enviados por el misma pares.

Protocolo de *handshake*

La mayoría de los mensajes intercambiados durante la configuración de la sesión TLS se basan en este registro, a menos que un error o advertencia se produzca y necesite ser señalado por un registro de protocolo de alerta (ver más abajo), o el modo de cifrado de la sesión es modificado por otro récord (véase el protocolo ChangeCipherSpec abajo).

+	Byte +0	Byte +1	Byte +2	Byte +3
Byte 0	22			
Bytes 1..4	Versión		Longitud	
	(Mayor)	(Menor)	(bits 15..8)	(bits 7..0)
Bytes 5..8	Tipo de Mensaje	Largo de los datos del mensaje de handshake		
		(bits 23..16)	(bits 15..8)	(bits 7..0)
Bytes 9..(n-1)	Datos del mensaje de handshake			
Bytes n..(n+3)	Tipo de Mensaje	Largo de los datos del mensaje de handshake		
		(bits 23..16)	(bits 15..8)	(bits 7..0)
Bytes (n+4)..	Datos del mensaje de handshake			

Tipo de Mensaje

Este campo identifica el tipo de mensaje del handshake.

Message Types	
Código	Descripción
0	HelloRequest
1	ClientHello
2	ServerHello
4	NewSessionTicket
11	Certificate
12	ServerKeyExchange
13	CertificateRequest
14	ServerHelloDone
15	CertificateVerify
16	ClientKeyExchange
20	Finished

Longitud de datos de mensajes de handshake

Este es un campo de 3 bytes que indica la longitud de los datos de handshake, no incluyendo la cabecera.

Tenga en cuenta que varios mensajes de *handshake* se pueden combinar dentro de un registro.

Protocolo de alertas

Este registro no debería enviarse normalmente durante los intercambios de protocolo de enlace o de aplicación normales. Sin embargo, este mensaje puede ser enviado en cualquier momento durante el *handshake* y hasta el cierre de la sesión. Si esto se utiliza para señalar un error fatal, la sesión se cerrará inmediatamente después de enviar este registro, por lo que este registro se utiliza para dar la razón de este cierre. Si el nivel de alerta se marca como una advertencia, el equipo remoto puede decidir cerrar la sesión si decide que la sesión no es lo suficientemente confiable para sus necesidades (antes de hacerlo, el remoto también puede enviar su propia señal).

+	Byte +0	Byte +1	Byte +2	Byte +3
Byte 0	21			
Bytes 1..4	Versión		Largo	
	(Mayor)	(Menor)	0	2
Bytes 5..6	Nivel	Descripción		
Bytes 7..(p-1)	MAC (opcional)			
Bytes p..(q-1)	Relleno (solo en cifrado de bloques)			

Nivel

Este campo identifica el nivel de alerta. Si el nivel es fatal, el remitente debe cerrar la sesión inmediatamente. De lo contrario, el destinatario puede decidir terminar la sesión, mediante el envío de su propia alerta fatal y cierre de la sesión misma inmediatamente después de enviarlo. El uso de registros de alerta es opcional, sin embargo si no se encuentra antes del cierre de sesión, la sesión puede reanudarse automáticamente (con sus handshake).

El cierre normal de la sesión después de la terminación de la aplicación transportada debe ser alertado preferiblemente con al menos el tipo de alerta *Close notify* (con un nivel de aviso sencillo) para evitar que la nueva sesión pueda ser reanudada. La señalización explícita del cierre normal de una sesión segura antes de cerrar de manera efectiva su capa de transporte es útil para prevenir o detectar los ataques (como los intentos de truncar los datos transportados de forma segura, si es que intrínsecamente no tiene una longitud predeterminada o duración que el destinatario de los datos puede esperar).

Tipos de Nivel de Alerta

Código	Tipo de Nivel	Estado de Conexión
1	advertencia	la conexión o la seguridad puede ser inestable.
2	fatal	la conexión o la seguridad puede estar comprometida, o un error irrecuperable ha ocurrido.

Descripción

Este campo identifica cual tipo de alerta está siendo enviada.

Descripción de tipos de Alertas

Código	Descripción	Tipos del Nivel	Notas
0	Notificación de Cierre	advertencia/fatal	
10	Mensaje Inesperado	fatal	
20	Mal registro MAC	fatal	Posiblemente una mala implementación de SSL, o la carga útil ha sido modificada con i.e. regla FTP de un firewall en un servidor FTPS.
21	Falla en Descifrado	fatal	solo TLS, reservado
22	Desbordamiento de Registro	fatal	solo TLS
30	Falla de Descompresión	fatal	
40	Falla de <i>handshake</i>	fatal	
41	Sin certificado	advertencia/fatal	Solo SSL 3.0, reservado
42	Certificado malo	advertencia/fatal	
43	Certificado no soportado	advertencia/fatal	por ej. el certificado tiene habilitado solo uso como autenticación de servidor y se presenta como certificado de cliente
44	Certificado revocado	advertencia/fatal	
45	Certificado expirado	advertencia/fatal	Revisar certificado del servidor expirado también revisar que algún certificado presentado en la cadena ha expirado
46	Certificado desconocido	advertencia/fatal	
47	parámetro ilegal	fatal	
48	CA (Autoridad de certificación) desconocida	fatal	solo TLS
49	Acceso denegado	fatal	solo TLS – por ej. no se ha presentado un certificado de cliente (TLS: mensaje de certificado en blanco o SSLv3: alerta de "No Certificado"), pero el servidor está configurado para requerir uno.
50	error de decodificación	fatal	solo TLS
51	error de descifrado	advertencia/fatal	solo TLS
60	Restricción de exportación	fatal	solo TLS, reservado
70	Versión de Protocolo	fatal	solo TLS
71	Seguridad Insuficiente	fatal	solo TLS
80	Error Interno	fatal	solo TLS
90	Cancelado por el Usuario	fatal	solo TLS
100	No renegociación	advertencia	solo TLS
110	Extensión No Soportada	advertencia	solo TLS
111	Certificado no Obtenible	advertencia	solo TLS
112	Nombre No Reconocido	advertencia/fatal	solo TLS; el Indicador de nombre del servidor del cliente especifica un host no soportado por el servidor
113	Respuesta de Mal Certificado	fatal	solo TLS
114	Valor de <i>hash</i> de Mal Certificado	fatal	solo TLS
115	Identidad PSK desconocida (usado en <u>TLS-PSK</u> y <u>TLS-</u>	fatal	solo TLS

	SRP)		
120	Sin Protocolo de Aplicación	fatal	solo TLS, el ALPN del cliente ALPN no contiene ningún protocolo soportado por el servidor.

Protocolo ChangeCipherSpec

+	Byte +0	Byte +1	Byte +2	Byte +3
Byte 0	20			
Bytes 1..4	Versión		Largo	
	(Mayor)	(Menor)	0	1
Byte 5	Tipo de Protocolo CCS			

Tipo de Protocolo CCS

Actualmente solo 1.

Protocolo de aplicación

+	Byte +0	Byte +1	Byte +2	Byte +3
Byte 0	23			
Bytes 1..4	Versión		Largo	
	(Mayor)	(Menor)	(bits 15..8)	(bits 7..0)
Bytes 5..(m-1)	Datos de Aplicación			
Bytes m..(p-1)	MAC (opcional)			
Bytes p..(q-1)	Relleno (solo cifrado de bloques)			

Largo

Largo de los datos de la aplicación (excluyendo el encabezado del protocolo e incluyendo el final de MAC y relleno)

MAC

20 bytes para el HMAC basado en SHA-1, 16 bytes para el HMAC basado en MD5.

Relleno

Largo variable; el último byte contiene el largo del relleno.

Soporte para servidores virtuales por nombre

Desde el punto de vista del protocolo de aplicaciones, TLS pertenece a una capa baja, aunque el modelo TCP/IP es muy grueso para mostrarlo. Esto significa que el *handshake* es usualmente (excepto en el caso STARTTLS) llevado a cabo antes de que el protocolo de aplicación pueda comenzar. La funcionalidad de servidor virtual basado en nombres es provista por la capa de aplicación, donde todos los servidores virtuales alojados en una misma máquina comparten el mismo certificado. Esto es un problema, ya que el servidor debe seleccionar y mandar el certificado inmediatamente después del mensaje de ClientHello. Este es un gran problema en los ambientes de alojamiento, ya que implica que todos los clientes en un mismo servidor deben compartir el certificado o se debe utilizar una IP distinta para cada uno de ellos. Hay dos formas conocidas de evitar esto, provistas por X.509:

- Si todos los servidores virtuales pertenecen al mismo dominio, se puede utilizar un certificado comodín. Además de que podría ser un problema la selección amplia de host ame, no hay acuerdo en cómo emparejar certificados wildcard. Se aplican reglas diferentes dependiendo del protocolo de aplicación o del software usado.¹⁰¹
- Agregar cada host virtual en la extensión *subjectAltName*. El problema mayor de esto es que el certificado necesita ser remitido cada vez que se agrega un nuevo servidor.

En orden a proveer el nombre del servidor, la RFC 4366 Transport Layer Security (TLS) Extensions permiten al cliente incluir una extensión de Indicación de nombre de servidor (Server Name Indication o SNI) en el mensaje extendido ClientHello. Esta extensión inmediatamente le da pistas al servidor respecto de cuál nombre es al que el cliente se quiere conectar, por lo que el servidor puede seleccionar el certificado apropiado para enviar al cliente.

Estándares

TLS 1.0

Las extensiones a TLS 1.0 incluyen:

- RFC 2595: "Using TLS with IMAP, POP3 and ACAP". Especifica una extensión a los servicios IMAP, POP3 y ACAP que permiten a cliente y servidor usar seguridad en la capa de transporte para entregar comunicaciones privadas y autenticadas sobre Internet.
- RFC 2712: "Addition of Kerberos Cipher Suites to Transport Layer Security (TLS)". Las familias de cifrados de 40-bit definidas en este memo aparecen sólo para propósitos de documentación del hecho de que esas familias de códigos de cifrado han sido ya asignadas.
- RFC 2817: "Upgrading to TLS Within HTTP/1.1", explica cómo usar el mecanismo Upgrade en HTTP/1.1 para iniciar TLS sobre una conexión TCP existente. Esto permite al tráfico HTTP inseguro y seguro compartir el mismo puerto *conocido* (en este caso, http: en el 80 en vez de https: en el 443).
- RFC 2818: "HTTP Over TLS", diferencia tráfico seguro de tráfico inseguro mediante el uso de un 'puerto de servidor' diferente.
- RFC 3207: "SMTP Service Extension for Secure SMTP over Transport Layer Security". Especifica una extensión al servicio SMTP que permiten a cliente y servidor SMTP usar seguridad en la capa de transporte para entregar comunicaciones privadas y autenticadas sobre Internet.
- RFC 3268: "AES Ciphersuites for TLS". Añade la familia de cifrado AES a los cifrados simétricos previamente existentes.
- RFC 3546: "Transport Layer Security (TLS) Extensions", añade un mecanismo para negociar extensiones de protocolos durante la inicialización de sesión y define algunas extensiones.
- RFC 3749: "Transport Layer Security Protocol Compression Methods", especifica el marco para los métodos de compresión y para el método DEFLATE.
- RFC 3943: "Transport Layer Security (TLS) Protocol Compression Using Lempel-Ziv-Stac (LZS)".
- RFC 4132: "Addition of Camellia Cipher Suites to Transport Layer Security (TLS)".
- RFC 4162: "Addition of SEED Cipher Suites to Transport Layer Security (TLS)".
- RFC 4217: "Securing FTP with TLS".
- RFC 4279: "Pre-Shared Key Ciphersuites for Transport Layer Security (TLS)", añade tres conjuntos de nuevas familias de cifrados para que el protocolo TLS permita la autenticación basada en claves previamente compartidas.

TLS 1.1

Las extensiones a TLS 1.1 incluyen:

- RFC 4347: "Datagram Transport Layer Security" especifica una variante de TLS que funciona sobre protocolos de datagramas (tales como UDP).
- RFC 4366: "Transport Layer Security (TLS) Extensions" describe tanto un set de extensiones específicas y un mecanismo de extensiones genéricas.
- RFC 4492: "Elliptic Curve Cryptography (ECC) Cipher Suites for Transport Layer Security (TLS)".
- RFC 4507: "Transport Layer Security (TLS) Session Resumption without Server-Side State".
- RFC 4680: "TLS Handshake Message for Supplemental Data".
- RFC 4681: "TLS User Mapping Extension".
- RFC 4785: "Pre-Shared Key (PSK) Ciphersuites with NULL Encryption for Transport Layer Security (TLS)".
- RFC 5054: "Using the Secure Remote Password (SRP) Protocol for TLS Authentication". Define las ciphersuites TLS-SRP.
- RFC 5081: "Using OpenPGP Keys for Transport Layer Security (TLS) Authentication", obsoleta después del RFC 6091.

TLS 1.2

La versión actual aprobada de TLS es la 1.2, la que se especifica en:

- RFC 5246: "The Transport Layer Security (TLS) Protocol Version 1.2".

El estándar actual reemplaza a las versiones más antiguas, las que son consideradas obsoletas:

- RFC 2246: "The TLS Protocol Version 1.0".

- [RFC 4346](#): “The Transport Layer Security (TLS) Protocol Version 1.1”.

así como el nunca estandarizado SSL 3.0:

- [RFC 6101](#): “The Secure Sockets Layer (SSL) Protocol Version 3.0”.

Otros [RFC](#) posteriores extendieron TLS.

Las extensiones a TLS 1.2 incluyen:

- [RFC 5746](#): “Transport Layer Security (TLS) Renegotiation Indication Extension”.
- [RFC 5878](#): “Transport Layer Security (TLS) Authorization Extensions”.
- [RFC 6091](#): “Using OpenPGP Keys for Transport Layer Security (TLS) Authentication”.
- [RFC 6176](#): “Prohibiting Secure Sockets Layer (SSL) Version 2.0”.
- [RFC 6209](#): “Addition of the ARIA Cipher Suites to Transport Layer Security (TLS)”.

TLS 1.3

- [RFC 8446](#), en inglés: *The Transport Layer Security (TLS) Protocol Version 1.3*.

Encapsulaciones

Las encapsulaciones de TLS incluyen:

- [RFC 5216](#): “The EAP-TLS Authentication Protocol”

Véase también

- [Datagram Transport Layer Security](#)
- [HTTPS](#)
- [X.509](#)
- [Wireless Transport Layer Security](#)
- [Heartbleed](#)
- [Network Security Services](#)] (NSS)

Referencias

- Dierks, T. y E. Rescorla (agosto de 2008). «The Transport Layer Security (TLS) Protocol, Version 1.2» (<https://tools.ietf.org/html/rfc5246>).
- «tls-vs-ssl» (<https://www.onos.com/digitalguide/server/security/tls-vs-ssl/>).
- «RFC 5246: The Transport Layer Security (TLS) Protocol Version 1.2» (<https://tools.ietf.org/html/rfc5246>). Internet Engineering Task Force. Consultado el 9 de septiembre de 2013.
- SSL: Intercepted today, decrypted tomorrow (<http://news.netcraft.com/archives/2013/06/25/ssl-intercepted-today-decrypted-tomorrow.html>), Netcraft, 2013-06-25.
- Freier, A., P. Karlton y P. Kocher (agosto de 2011). «The Secure Sockets Layer (SSL) Protocol Version 3.0» (<https://tools.ietf.org/html/rfc6101>).
- Kovacs, Eduard (13 de agosto de 2018). «IETF Publishes TLS 1.3 as RFC 8446» (<https://www.securityweek.com/ietf-publishes-tls-13-rfc-8446>). *securityweek.com* (en inglés). Consultado el 16 de agosto de 2018.
- Woo, Thomas Y. C., Raghuram Bindignavle, Shaowen Su y Simon S. Lam, *SNP: An interface for secure network programming* Proceedings USENIX Summer Technical Conference, June 1994
- «THE SSL PROTOCOL» (<https://web.archive.org/web/19970614020952/http://home.netscape.com/newsref/std/SSL.html>). Netscape Corporation. 2007. Archivado desde el original (<http://home.netscape.com/newsref/std/SSL.html>) el 14 de junio de 1997. Consultado el 13 de enero de 2013.
- Rescorla 2001
- Dierks, T. y E. Rescorla (abril de 2006). «The Transport Layer Security (TLS) Protocol Version 1.1, RFC 4346» (<https://tools.ietf.org/html/rfc5246#ref-TLS1.1>).
- [RFC 6091](#): “Using OpenPGP Keys for Transport Layer Security (TLS) Authentication”
- Eronen, P. (ed.). «RFC 4279: Pre-Shared Key Ciphersuites for Transport Layer Security (TLS)» (<https://tools.ietf.org/html/rfc4279>). Internet Engineering Task Force. Consultado el 9 de septiembre de 2013.
- Gothard, Peter. «Google updates SSL certificates to 2048-bit encryption» (<https://web.archive.org/web/20130922082322/http://www.computing.co.uk/ctg/news/2285984/google-updates-ssl-certificates-to-2048bit-encryption>). *Computing*. Incisive Media. Archivado desde el original (<http://www.computing.co.uk/ctg/news/2285984/google-u>

- pdates-ssl-certificates-to-2048bit-encryption) el 22 de septiembre de 2013. Consultado el 9 de septiembre de 2013.
14. draft-chudov-cryptopro-cp1s-04 - GOST 28147-89 Cipher Suites for Transport Layer Security (TLS) (<https://tools.ietf.org/html/draft-chudov-cryptopro-cp1s-04>) (en inglés)
 15. RFC 8446
 16. RFC 5288
 17. RFC 6655
 18. RFC 6367
 19. RFC 5932
 20. RFC 4162
 21. «On the Practical (In-)Security of 64-bit Block Ciphers — Collision Attacks on HTTP over TLS and OpenVPN» (http://sweet32.info/SWEET32_CCS16.pdf). 28 de octubre de 2016. Archivado (https://web.archive.org/web/20170424021101/https://sweet32.info/SWEET32_CCS16.pdf) desde el original el 24 de abril de 2017. Consultado el 8 de junio de 2017.
 22. «NIST Special Publication 800-57 *Recommendation for Key Management — Part 1: General (Revised)*» (https://web.archive.org/web/20140606050814/http://csrc.nist.gov/publications/nistpubs/800-57/sp800-57-Part1-revised2_Mar08-2007.pdf). 8 de marzo de 2007. Archivado desde el original (http://csrc.nist.gov/publications/nistpubs/800-57/sp800-57-Part1-revised2_Mar08-2007.pdf) el 6 de junio de 2014. Consultado el 3 de julio de 2014.
 23. Qualys SSL Labs. «SSL/TLS Deployment Best Practices» (<https://www.ssllabs.com/projects/best-practices/index.html>). Archivado (<https://web.archive.org/web/20150704101956/https://www.ssllabs.com/projects/best-practices/index.html>) desde el original el 4 de julio de 2015. Consultado el 2 de junio de 2015.
 24. IETF RFC 5469
 25. draft-agl-tls-chacha20poly1305-04 (<https://tools.ietf.org/html/draft-agl-tls-chacha20poly1305-04>)
 26. Al 6 de noviembre de 2014. «SSL Pulse: Survey of the SSL Implementation of the Most Popular Web Sites» (<https://web.archive.org/web/20170515034337/https://www.trustworthyinternet.org/ssl-pulse/>). Archivado desde el original (<https://www.trustworthyinternet.org/ssl-pulse/>) el 15 de mayo de 2017. Consultado el 11 de noviembre de 2014.
 27. ivanr. «RC4 in TLS is Broken: Now What?» (<https://community.qualys.com/blogs/securitylabs/2013/03/19/rc4-in-tls-is-broken-now-what>). Qualys Security Labs. Consultado el 30 de julio de 2013.
 28. Möller, Bodo; Thai Duong y Krzysztof Kotowicz. «This POODLE Bites: Exploiting The SSL 3.0 Fallback» (<https://www.openssl.org/~bodo/ssl-poodle.pdf>). Consultado el 15 de octubre de 2014.
 29. Google (29 de mayo de 2012). «Dev Channel Update» (http://googlechromereleases.blogspot.nl/2012/05/dev-channel-update_29.html). Consultado el 14 de enero de 2013.
 30. Google (21 de agosto de 2012). «Stable Channel Update» (http://googlechromereleases.blogspot.co.uk/2012/08/stable-channel-update_21.html). Consultado el 14 de enero de 2013.
 31. «Security in Firefox 2» (https://web.archive.org/web/20120723191042/https://developer.mozilla.org/en/Security_in_Firefox_2). 6 de agosto de 2008. Archivado desde el original (https://developer.mozilla.org/en/Security_in_Firefox_2) el 23 de julio de 2012. Consultado el 14 de enero de 2013.
 32. «Bug 733647 - Implement TLS 1.1 (RFC 4346) in Gecko (Firefox, Thunderbird), on by default» (https://bugzilla.mozilla.org/show_bug.cgi?id=733647). 6 de marzo de 2012. Consultado el 14 de enero de 2013.
 33. «Bug 480514 - Implement support for TLS 1.2 (RFC 5246)» (https://bugzilla.mozilla.org/show_bug.cgi?id=480514). 17 de marzo de 2010. Consultado el 14 de enero de 2013.
 34. Microsoft (5 de septiembre de 2012). «Secure Channel» ([http://msdn.microsoft.com/en-us/library/windows/desktop/aa380123\(v=vs.85\).aspx](http://msdn.microsoft.com/en-us/library/windows/desktop/aa380123(v=vs.85).aspx)). Consultado el 14 de enero de 2013.
 35. Microsoft (27 de febrero de 2009). «MS-TLSP Appendix A» (<http://msdn.microsoft.com/en-us/library/dd208005%28PROT.13%29.aspx>). Consultado el 14 de enero de 2013.
 36. Pettersen, Yngve Nysæter (25 de febrero de 2009). «New in Opera Presto 2.2: TLS 1.2 Support» (<https://web.archive.org/web/20090304164942/http://my.opera.com/core/blog/2009/02/25/new-in-opera-presto-2-2-tls-1-2-support>). Archivado desde el original (<http://my.opera.com/core/blog/2009/02/25/new-in-opera-presto-2-2-tls-1-2-support>) el 4 de marzo de 2009. Consultado el 14 de enero de 2013.
 37. Adrian, Dimcev. «Common browsers/libraries/servers and the associated cipher suites implemented» (http://www.carbonwind.net/TLS_Cipher_Suites_Project/tls_ssl_cipher_suites_annex_a1_main.docx). *TLS Cipher Suites Project*.
 38. Apple (10 de junio de 2009). «Features» (<http://www.apple.com/safari/features.html>). Consultado el 14 de enero de 2013.
 39. Apple (14 de octubre de 2011). «Technical Note TN2287 - iOS 5 and TLS 1.2 Interoperability Issues» (https://developer.apple.com/library/ios/#technotes/tn2287/_index.html). Consultado el 14 de enero de 2013.
 40. Liebowitz, Matt (13 de octubre de 2011). «Apple issues huge software security patches» (https://web.archive.org/web/20111128142441/http://www.msnbc.msn.com/id/44896639/ns/technology_and_science-security/t/apple-issues-huge-software-security-patches/). *NBCNews.com*. Archivado desde el original (https://www.msnbc.msn.com/id/44896639/ns/technology_and_science-security/t/apple-issues-huge-software-security-patches/) el 28 de noviembre de 2011. Consultado el 14 de enero de 2013.
 41. MWR Info Security (16 de abril de 2012). «Adventures with iOS UIWebviews» (<https://web.archive.org/web/20130320091836/http://labs.mwrinfosecurity.com/blog/2012/04/16/adventures-with-ios-uiwebviews/>). Archivado desde el original (<http://labs.mwrinfosecurity.com/blog/2012/04/16/adventures-with-ios-uiwebviews/>) el 20 de marzo de 2013. Consultado el 14 de enero de 2013.,

section "HTTPS (SSL/TLS)"

42. Georgiev, Martin; Iyengar, Subodh; Jana, Suman; Anubhai, Rishita; Boneh, Dan y Shmatikov, Vitaly (2012). «The most dangerous code in the world: validating SSL certificates in non-browser software.» (http://www.cs.uteexas.edu/~shmat/shmat_ccs12.pdf). *Proceedings of the 2012 ACM conference on Computer and communications security*. pp. 38-49. ISBN 978-1-4503-1651-4.
43. Claessens, Joris; Valentin Dem, Danny De Cock, Bart Preneel, Joos Vandewalle (2002). «On the Security of Today's Online Electronic Banking Systems» (<http://www.sciencedirect.com/science/article/pii/S0167404802003127>). *Computers & Security* **21** (3): 253-265. doi:10.1016/S0167-4048(02)00312-7 (<https://dx.doi.org/10.1016%2FS0167-4048%2802%2900312-7>).
44. Lawrence, Eric (22 de octubre de 2005). «IEBlog: Upcoming HTTPS Improvements in Internet Explorer 7 Beta 2» (<http://blogs.msdn.com/ie/archive/2005/10/22/483795.aspx>). MSDN Blogs. Consultado el 25 de noviembre de 2007.
45. «Bugzilla@Mozilla — Bug 236933 - Disable SSL2 and other weak ciphers» (https://bugzilla.mozilla.org/show_bug.cgi?id=236933). Mozilla Corporation. Consultado el 25 de noviembre de 2007.
46. "Opera 9.5 for Windows Changelog" (<http://www.opera.com/docs/changelogs/windows/950/>) Archivado (<https://web.archive.org/web/20090626142453/http://www.opera.com/docs/changelogs/windows/950/>) el 26 de junio de 2009 en Wayback Machine. at Opera.com: "Disabled SSL v2 and weak ciphers."
47. «Firefox still sends SSLv2 handshake even though the protocol is disabled» (https://bugzilla.mozilla.org/show_bug.cgi?id=454759). 11 de septiembre de 2008.
48. "Opera 10 for Windows changelog" (<http://www.opera.com/docs/changelogs/windows/1000/>) en Opera.com: "Removed support for SSL v2 and weak ciphers"
49. Pettersen, Yngve (30 de abril de 2007). «10 years of SSL in Opera — Implementer's notes» (<https://web.archive.org/web/20070506145847/http://my.opera.com/yngve/blog/2007/04/30/10-years-of-ssl-in-opera>). Opera Software. Archivado desde el original (<http://my.opera.com/yngve/blog/2007/04/30/10-years-of-ssl-in-opera>) el 6 de mayo de 2007. Consultado el 25 de noviembre de 2007.
50. National Institute of Standards and Technology (diciembre de 2010). «Implementation Guidance for FIPS PUB 140-2 and the Cryptographic Module Validation Program» (<http://csrc.nist.gov/groups/STM/cmvp/documents/fips140-2/FIPS1402IG.pdf>) (en inglés).
51. DanGoodin (19 de septiembre de 2011). «Hackers break SSL encryption used by millions of sites» (http://www.theregister.co.uk/2011/09/19/beast_exploits_paypal_ssl/).
52. «Y Combinator comments on the issue» (<http://news.ycombinator.com/item?id=3015498>). 20 de septiembre de 2011.
53. Dimcev, Adrian. «SSL/TLS version rollbacks and browsers» (<https://web.archive.org/web/20110320040216/http://www.carbonwind.net/blog/post/Random-SSLTLS-101%E2%80%933SSLTLS-version-rollbacks-and-browser>). Archivado desde el original (<http://www.carbonwind.net/blog/post/Random-SSLTLS-101%E2%80%933SSLTLS-version-rollbacks-and-browsers.aspx>) el 20 de marzo de 2011. Consultado el 9 de marzo de 2011.
54. Rescorla, Eric (5 de noviembre de 2009). «Understanding the TLS Renegotiation Attack» (https://web.archive.org/web/20120211120608/http://www.educatedguesswork.org/2009/11/understanding_the_tls_renegoti.html). *Educated Guesswork*. Archivado desde el original (http://www.educatedguesswork.org/2009/11/understanding_the_tls_renegoti.html) el 11 de febrero de 2012. Consultado el 27 de noviembre de 2009.
55. «SSL_CTX_set_options SECURE_RENEGOTIATION» (http://www.webcitation.org/65JSTpBgx?url=http://www.openssl.org/docs/ssl/SSL_CTX_set_options.html#SECURE_RENEGOTIATION). *OpenSSL Docs*. 25 de febrero de 2010. Archivado desde el original (https://www.openssl.org/docs/ssl/SSL_CTX_set_options.html#SECURE_RENEGOTIATION) el 9 de febrero de 2012. Consultado el 18 de noviembre de 2010.
56. «GnuTLS 2.10.0 released» (<https://www.webcitation.org/65JSUYEDA?url=http://article.gmane.org/gmane.network.gnutls.general/2046>). *GnuTLS release notes*. 25 de junio de 2010. Archivado desde el original (<http://article.gmane.org/gmane.network.gnutls.general/2046>) el 9 de febrero de 2012. Consultado el 24 de julio de 2011.
57. «NSS 3.12.6 release notes» (https://web.archive.org/web/20110629112326/https://developer.mozilla.org/NSS_3.12.6_release_notes). *NSS release notes*. 3 de marzo de 2010. Archivado desde el original (https://developer.mozilla.org/NSS_3.12.6_release_notes) el 29 de junio de 2011. Consultado el 24 de julio de 2011.
58. Langley, A.; N. Modadugu, B. Moeller (June de 20120). «Transport Layer Security (TLS) False Start» (<https://tools.ietf.org/html/draft-bmoeller-tls-falsestart-00>). *Internet Engineering Task Force*. IETF. Consultado el 31 de julio de 2013.
59. Wolfgang, Gruener. «False Start: Google Proposes Faster Web, Chrome Supports It Already» (<https://web.archive.org/web/20101007061707/http://www.conceivablytech.com/3299/products/false-start-google-proposes-faster-web-chrome-supports-it-already/>) el 7 de octubre de 2010. Consultado el 9 de marzo de 2011.
60. Brian, Smith. «Limited rollback attacks in False Start and Snap Start» (<http://www.ietf.org/mail-archive/web/tls/current/msg06933.html>). Consultado el 9 de marzo de 2011.
61. Adrian, Dimcev. «False Start» (<https://web.archive.org/web/20110504060256/http://www.carbonwind.net/blog/post/Random-SSLTLS-101-False-Start.aspx>). *Random SSL/TLS 101*. Archivado desde el original (<http://www.carbonwind.net/blog/post/Random-SSLTLS-101-False-Start.aspx>) el 4 de mayo de 2011. Consultado el 9 de marzo de 2011.
62. Mavrogiannopoulos, Nikos; Vercautern, Frederik; Velichkov, Vesselin y Preneel, Bart (2012). «A cross-protocol attack on the TLS protocol.» (<https://www.cosic.esat.kuleuven.be/publications/article-2216.pdf>).

- Proceedings of the 2012 ACM Conference on Computer and Communications Security*. pp. 62-72. ISBN 978-1-4503-1651-4.
63. «Security of CBC Ciphersuites in SSL/TLS» (<https://web.archive.org/web/20120630143111/http://www.openssl.org/~bodo/tls-cbc.txt>). 20 de mayo de 2004. Archivado desde el original (<https://www.openssl.org/~bodo/tls-cbc.txt>) el 30 de junio de 2012. Consultado el 13 de enero de 2013.
 64. Smith, Brian (30 de septiembre de 2011). «(CVE-2011-3389) Rizzo/Duong chosen plaintext attack (BEAST) on SSL/TLS 1.0 (facilitated by websockets -76)» (https://bugzilla.mozilla.org/show_bug.cgi?id=665814).
 65. «Vulnerability in SSL/TLS Could Allow Information Disclosure (2643584)» (<http://technet.microsoft.com/en-us/security/bulletin/ms12-006>). 10 de enero de 2012.
 66. Ristic, Ivan (31 de octubre de 2013). «Apple Enabled BEAST Mitigations in OS X 10.9 Mavericks» (<https://community.qualys.com/blogs/securitylabs/2013/10/31/apple-enabled-beast-mitigations-in-os-x-109-mavericks>). *Security Labs* (en inglés). Consultado el 1 de febrero de 2014.
 67. «Safest ciphers to use with the BEAST? (TLS 1.0 exploit)» (<http://serverfault.com/questions/315042/>). 24 de septiembre de 2011.
 68. «First solutions for SSL/TLS vulnerability» (<http://www.h-online.com/security/news/item/First-solutions-for-SSL-TLS-vulnerability-1349813.html>). 26 de septiembre de 2011.
 69. Goodin, Dan (13 de septiembre de 2012). «Crack in Internet's foundation of trust allows HTTPS session hijacking» (<http://arstechnica.com/security/2012/09/crim-e-hijacks-https-sessions/>). Ars Technica. Consultado el 31 de julio de 2013.
 70. Fisher, Dennis (13 de septiembre de 2012). «CRIME Attack Uses Compression Ratio of TLS Requests as Side Channel to Hijack Secure Sessions» (https://web.archive.org/web/20120915224635/http://threatpost.com/en_us/blogs/crime-attack-uses-compression-ratio-tls-requests-side-channel-hijack-secure-sessions-091312). ThreatPost. Archivado desde el original (http://threatpost.com/en_us/blogs/crime-attack-uses-compression-ratio-tls-requests-side-channel-hijack-secure-sessions-091312) el 15 de septiembre de 2012. Consultado el 13 de septiembre de 2012.
 71. Gutmann, P. (septiembre de 2014). «Encrypt-then-MAC for Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)» (<https://tools.ietf.org/html/rfc7366>).
 72. Möller, Bodo (14 de octubre de 2014). «This POODLE bites: exploiting the SSL 3.0 fallback» (<http://googleonlinesecurity.blogspot.com/2014/10/this-poodle-bites-exploiting-ssl-30.html>). Consultado el 15 de octubre de 2014.
 73. Bar-El, Hagai. «Poodle flaw and IoT» (<http://www.hbarel.com/analysis/eng/poodle-flaw-and-iot>). Consultado el 15 de octubre de 2014.
 74. security – Safest ciphers to use with the BEAST? (TLS 1.0 exploit) I've read that RC4 is immune – Server Fault (<http://serverfault.com/questions/315042/>) (en inglés)
 75. Pouyan Sepehrdad, Serge Vaudenay, Martin Vuagnoux (2011). «Discovery and Exploitation of New Biases in RC4» (http://link.springer.com/chapter/10.1007%2F978-3-642-19574-7_5). *Lecture Notes in Computer Science* **6544**: 74-91. doi:10.1007/978-3-642-19574-7_5 (https://dx.doi.org/10.1007%2F978-3-642-19574-7_5).
 76. Green, Matthew. «Attack of the week: RC4 is kind of broken in TLS» (<http://blog.cryptographyengineering.com/2013/03/attack-of-week-rc4-is-kind-of-broken-in.html>). *Cryptography Engineering*. Consultado el 12 de marzo de 2013.
 77. AlFardan, Nadhem; Dan Bernstein, Kenny Paterson, Bertram Poettering y Jacob Schuldt. «On the Security of RC4 in TLS» (<http://www.isg.rhul.ac.uk/tls/>). Royal Holloway University of London. Consultado el 13 de marzo de 2013.
 78. AlFardan, Nadhem J.; Bernstein, Daniel J.; Paterson, Kenneth G.; Poettering, Bertram; Schuldt, Jacob C. N. (8 de julio de 2013). *On the Security of RC4 in TLS and WPA* (<https://web.archive.org/web/20130922170155/http://www.isg.rhul.ac.uk/tls/RC4biases.pdf>) (PDF). Archivado desde el original (<http://www.isg.rhul.ac.uk/tls/RC4biases.pdf>) el 22 de septiembre de 2013. Consultado el 2 de septiembre de 2013.
 79. AlFardan, Nadhem J.; Bernstein, Daniel J.; Paterson, Kenneth G.; Poettering, Bertram; Schuldt, Jacob C. N. (15 de agosto de 2013). *On the Security of RC4 in TLS* (https://www.usenix.org/sites/default/files/conference/protected-files/alfardan_sec13_slides.pdf) (PDF). 22nd USENIX Security Symposium. p. 51. Consultado el 2 de septiembre de 2013. «Plaintext recovery attacks against RC4 in TLS are feasible although not truly practical».
 80. Popov, Andrei (1 de octubre de 2014). «Prohibiting RC4 Cipher Suites - draft-ietf-tls-prohibiting-rc4-01» (<https://tools.ietf.org/html/draft-ietf-tls-prohibiting-rc4-01>).
 81. Leyden, John (1 de agosto de 2013). «Gmail, Outlook.com and e-voting 'pwned' on stage in crypto-dodge hack» (http://www.theregister.co.uk/2013/08/01/gmail_hotmail_hijacking/). *The Register*. Consultado el 1 de agosto de 2013.
 82. «BlackHat USA Briefings» (<https://www.blackhat.com/us-13/briefings.html#Smyth>). *Black Hat 2013*. Consultado el 1 de agosto de 2013.
 83. «Why is it called the 'Heartbleed Bug'?» (<http://www.washingtonpost.com/blogs/style-blog/wp/2014/04/09/why-is-it-called-the-heartbleed-bug/>).
 84. «Heartbleed Bug vulnerability [9 April 2014]» (<https://web.archive.org/web/20140705212748/https://blogs.comodo.com/e-commerce/heartbleed-bug-comodo-urges-open-ssl-users-to-apply-patch/>). Archivado desde el original (<https://blogs.comodo.com/e-commerce/heartbleed-bug-comodo-urges-open-ssl-users-to-apply-patch/>) el 5 de julio de 2014. Consultado el 25 de noviembre de 2014.
 85. Diffie, Whitfield; van Oorschot, Paul C.; Wiener, Michael J. (Junio de 1992). «Authentication and Authenticated Key Exchanges» (<http://citeseer.ist.psu.edu/diffie92authentication.html>). *Designs, Codes and Cryptography* **2** (2): 107-125. doi:10.1007/BF00124891 (<https://dx.doi.org/10.1007%2FBF00124891>). Consultado el 11 de febrero de 2008.
 86. «Discussion on the TLS mailing list in October 2007» (<https://web.archive.org/web/20130922103746/http://www.i>

- etf.org/mail-archive/web/tls/current/msg02134.html). Archivado desde el original (<http://www1.ietf.org/mail-archive/web/tls/current/msg02134.html>) el 22 de septiembre de 2013. Consultado el 3 de enero de 2015.
87. «Protecting data for the long term with forward secrecy» (<http://googleonlinesecurity.blogspot.com.au/2011/11/protecting-data-for-long-term-with.html>). Consultado el 5 de noviembre de 2012.
88. Bernat, Vincent. «SSL/TLS & Perfect Forward Secrecy» (<https://vincent.bernat.im/en/blog/2011-ssl-perfect-forward-secrecy.html>). Consultado el 5 de noviembre de 2012.
89. «SSL Labs: Deploying Forward Secrecy» (<https://community.qualys.com/blogs/securitylabs/2013/06/25/ssl-labs-deploying-forward-secrecy>). Qualys.com. 25 de junio de 2013. Consultado el 10 de julio de 2013.
90. Ristic, Ivan (5 de agosto de 2013). «SSL Labs: Deploying Forward Secrecy» (<https://community.qualys.com/blogs/securitylabs/2013/06/25/ssl-labs-deploying-forward-secrecy>). Qualys. Consultado el 31 de agosto de 2013.
91. Langley, Adam (27 de junio de 2013). «How to botch TLS forward secrecy» (<https://www.imperialviolet.org/2013/06/27/botchingpfs.html>).
92. Daignière, Florent. «TLS "Secrets": Whitepaper presenting the security implications of the deployment of session tickets (RFC 5077) as implemented in OpenSSL» (<https://media.blackhat.com/us-13/US-13-Daigniere-TLS-Secrets-WP.pdf>). Matta Consulting Limited. Consultado el 7 de agosto de 2013.
93. Daignière, Florent. «TLS "Secrets": What everyone forgot to tell you...» (<https://media.blackhat.com/us-13/US-13-Daigniere-TLS-Secrets-Slides.pdf>). Matta Consulting Limited. Consultado el 7 de agosto de 2013.
94. «Protecting data for the long term with forward secrecy» (<http://googleonlinesecurity.blogspot.com.au/2011/11/protecting-data-for-long-term-with.html>). Consultado el 7 de marzo de 2014.
95. Hoffman-Andrews, Jacob. «Forward Secrecy at Twitter» (<https://blog.twitter.com/2013/forward-secrecy-at-twitter-0>). Twitter. Consultado el 7 de marzo de 2014.
96. "Certificate Pinning" (<https://viaforensics.com/resources/reports/best-practices-ios-android-secure-mobile-development/41-certificate-pinning/>) Archivado (<https://web.archive.org/web/20131227224656/https://viaforensics.com/resources/reports/best-practices-ios-android-secure-mobile-development/41-certificate-pinning/>) el 27 de diciembre de 2013 en Wayback Machine..
97. "Fijación de clave pública lanzado en Firefox" (<https://blog.mozilla.org/security/2014/09/02/public-key-pinning/>)
98. Proyecto Perspectivas (<http://perspectives-project.org/>)
99. Estos certificados son actualmente X.509, pero el RFC 6091 también especifica el uso de certificados basados en OpenPGP.
100. Chris (18 de febrero de 2009). «vsftpd-2.1.0 released - Using TLS session resume for FTPS data connection authentication» (<http://scarybeastsecurity.blogspot.com/2009/02/vsftpd-210-released.html>). Scarybeastsecurity.blogspot.com. Consultado el 17 de mayo de 2012.
101. «Named-based SSL virtual hosts: how to tackle the problem» (https://www.switch.ch/pki/meetings/2007-01/namebased_ssl_virtualhosts.pdf) (PDF). Consultado el 17 de mayo de 2012.

Bibliografía

- Wagner, David y Bruce Schneier, «Analysis of the SSL 3.0 Protocol», *The Second USENIX Workshop on Electronic Commerce Proceedings*, USENIX Press, November 1996, pp. 29-40.

Enlaces externos

- Centro de información de SSL (<http://www.verisign.es/products-services/security-services/ssl/ssl-information-center/index.html>)
- El grupo de trabajo de la IETF de TLS (<https://web.archive.org/web/20060206141643/http://www.ietf.org/html.charters/tls-charter.html>) (en inglés)
- SSL: Guía de generación de CSR (<http://www.hispassl.com/soporte/>)
- SSL: Guía SSL wildcard CSR (<https://ciberseguridadinternet.com/>)
- Problemas de seguridad existentes en SSL/TLS (https://web.archive.org/web/20130521034521/http://www.hacktimes.com/certificados_ssl_tls/)
- SSL: CSR generation guide (https://web.archive.org/web/20130731012219/http://www.digitar.net/services/ssl/ssl/ssl_guides.php) Instructivos de instalación de Certificados SSL ordenados por plataforma
- Test de configuración de SSL (<https://www.ssllabs.com/ssltest/>)
- Guía de Instalación de SSL (<https://web.archive.org/web/20161220225907/https://www.catar.co/certificado-ssl-gratis-con-lets-encrypt>)
- Cómo generar CSR para SSL (<https://www.essl.co/how-to-generate-csr/>)
- Preguntas frecuentes de Certificados SSL (<https://www.grupo.host/edu/c/certificados-ssl>)